

HTL Maturatrainer

DIPLOMARBEIT

verfasst im Rahmen der

Reife- und Diplomprüfung

an der

Höheren Abteilung für Medientechnik

Eingereicht von:

Thomas Gossenreiter
Stefanie Steinmair

Betreuer:

Birgit Schröder

Leonding, 6. April 2026

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt bzw. die wörtlich oder sinngemäß entnommenen Stellen als solche kenntlich gemacht habe.

Die Arbeit wurde bisher in gleicher oder ähnlicher Weise keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Die vorliegende Diplomarbeit ist mit dem elektronisch übermittelten Textdokument identisch.

Leonding, 6. April 2026

Thomas Gossenreiter & Stefanie Steinmair

Abstract

This diploma thesis focuses on the development of a web-based learning platform designed to support students in preparing for their final school leaving examinations (Matura). The objective of this work is to provide students with individualized learning support and to assist them in building knowledge through a structured learning strategy and realistic exam simulations.



The core focus lies on the clear separation of learning and testing phases. This is achieved through a practice mode featuring automated repetition logic as well as an exam mode with time limits and delayed feedback. Furthermore, the application is enhanced by statistical evaluations, which provide transparent tracking of learning progress and a targeted analysis of errors through integrated solution paths.

Zusammenfassung

Diese Diplomarbeit beschäftigt sich mit der Entwicklung einer webbasierten Lernplattform zur Vorbereitung auf die Reifeprüfung (Matura). Ziel dieser Arbeit ist, Schüler:innen eine individuelle Lernbegleitung zu bieten und durch eine strukturierte Lernstrategie und durch realitätsnahe Prüfungssimulationen beim Wissensaufbau zu unterstützen.



Der Fokus liegt dabei auf der Trennung von Lern- und Testphasen. Dies wird durch einen Übungsmodus mit automatisierter Wiederholungslogik sowie einen Prüfungsmodus mit Zeitlimit und verzögertem Feedback umgesetzt. Die Anwendung wird außerdem durch statistische Auswertungen ergänzt, welche eine transparente Verfolgung des Lernfortschritts und eine gezielte Analyse von Fehlern durch hinterlegte Lösungswege bietet.

Inhaltsverzeichnis

1	Einleitung	1
1.1	Ausgangslage	1
1.2	Motivation und Problemstellung	1
1.3	Grundlagen und Analyse	2
1.3.1	Marktanalyse	2
1.3.2	Maturatraining digitalisieren	2
1.4	Lernstrategien	3
1.4.1	Leitner-System	3
1.4.2	Spaced Repetition	4
1.4.3	Microlearning	5
1.4.4	Vergleich verschiedener Lernstrategien und Schlussfolgerung . .	6
1.5	Zielsetzung	7
2	Technologische Entscheidungen und Architektur	9
2.1	Backend	9
2.1.1	Spring Boot – Die Evolution des Java-Enterprise-Standards . . .	10
2.1.2	Node.js - Das ereignisgesteuerte Gegenmodell	14
2.1.3	Quarkus	19
2.1.4	Fazit und finale Technologieentscheidung	24
2.2	Frontend	25
2.2.1	Angular	27
2.2.2	Flutter	29
2.2.3	Ergebnis	31
3	Praktische Umsetzung	34
3.1	Logische Programmstruktur und Use Cases	34
3.2	Use Case: Üben	35
3.2.1	Verwaltung von Fragenpools	36

3.2.2	Fragenhandling und Fragetypen	40
3.2.3	Übungsmodus mit direktem Feedback	42
3.2.4	Lösungswege und Nutzerinteraktion	46
3.3	Use Case: Prüfungssimulation	47
3.3.1	Konfiguration einer Prüfung	47
3.3.2	Prüfungsmodus mit Zeitlimit	49
3.3.3	Bewertung und Ergebnisausgabe	51
3.4	Use Case: Fortschritt einsehen & analysieren	54
3.4.1	Lernstrategien und Fehleranalyse	55
3.4.2	Spaced Repetition nach Leitner	56
3.4.3	Lernstatistiken und Visualisierung	58
3.5	Implementierungsdetails	59
3.5.1	Datenmodell	59
3.5.2	Überblick über Komponenten	64
3.5.3	Custom CSS-Klassen und eingebundene Bibliotheken	66
4	Zusammenfassung	71
	Literaturverzeichnis	V
	Abbildungsverzeichnis	VII
	Tabellenverzeichnis	VIII

1 Einleitung

1.1 Ausgangslage

Die Vorbereitung auf Tests, Schularbeiten und schließlich die Matura erfordern ein hohes Maß an Eigenverantwortung und Organisation. Im Moment erfolgt das Lernen größtenteils mithilfe von Skripten, Arbeitsblättern und Übungsbeispielen aus dem Unterricht. Diese bewährten Methoden stoßen allerdings dort an ihre Grenzen, wo es um die Dynamik des Lernprozess geht.

Zwar ist eine erfolgreiche Vorbereitung mit den vorhandenen Mitteln durchaus möglich, sie ist allerdings häufig mit einem hohen organisatorischen Aufwand verbunden. Wesentlich dabei ist das fehlende unmittelbare Feedback beim eigenständigen Üben. Schüler und Schülerinnen können ihre Ergebnisse mit Musterlösungen abgleichen, dabei bleibt jedoch der Weg zur Lösung und die Analyse spezifischer Fehlerquellen oft ungeklärt. Weiters ist es ohne digitale Unterstützung schwierig, den individuellen Lernfortschritt über einen längeren Zeitraum präzise zu verfolgen oder gezielt Schwachstellen zu identifizieren, die mehr Übung erfordern.

Bereits bestehende Lernplattformen sind oft eher allgemein gehalten und erfüllen die spezifischen Anforderungen des technischen Lehrplans nur bedingt. Die geplante Matura-Lernplattform setzt genau hier an. Sie soll nicht als Ersatz für den Unterricht, sondern als ergänzendes Werkzeug dienen und den Lernvorgang für Schüler:innen durch direktes Feedback und eine strukturierte Fortschrittskontrolle effizienter und transparenter gestalten.

1.2 Motivation und Problemstellung

Die im vorherigen Abschnitt beschriebene Ausgangssituation zeigt, dass der Lernprozess bei der Vorbereitung für die Matura stark von eigenständigem Üben und individueller Organisation geprägt ist. Die Motivation für diese Diplomarbeit ergibt sich aus der Überlegung, wie moderne Webtechnologien genutzt werden können, um diesen Prozess von einer passiven Kontrolle, wie es das reine Abgleichen mit Musterlösungen ist, hin

zu einer aktiven Lernbegleitung zu transformieren. Daraus ergibt sich eine zentrale Fragestellung: **Wie kann eine digitale Plattform gestaltet sein, sodass sie den Lernprozess strukturierter, transparenter und effizienter macht?**

Ein wesentlicher Aspekt liegt hierbei in der Handhabung von Fehlern. Im aktuellen Lernvorgang werden Fehler oft nur zur Kenntnis genommen, nicht aber systematisch für den weiteren Lernpfad genutzt. Es bleibt also oft unklar, ob ein Fehler auf einem flüchtigen Rechenfehler oder einem grundlegenden Verständnisproblem basiert. Die Motivation dieser Arbeit liegt darin, ein System zu entwickeln, das genau solche Schwachstellen identifiziert und durch gezielte Wiederholungsintervalle den Aufbau eines nachhaltigen Wissensfundaments unterstützt.

Zusätzlich stellt auch der Mangel an Transparenz über den eigenen Wissensstand eine Motivation für die Entwicklung an diesem Projekt dar. Während klassische Lernmaterialien statisch sind, bietet eine digitale Plattform die Chance, den eigenen Lernfortschritt über Monate hinweg zu sammeln und zu verfolgen. Ziel ist es, den Schüler:innen ein Werkzeug zur Verfügung zu stellen, welches durch Visualisierung von Lernstatistiken die Selbsteinschätzung unterstützt und somit die organisatorische Last der Prüfungsvorbereitung reduziert.

1.3 Grundlagen und Analyse

1.3.1 Marktanalyse

Analyse bestehender Lernplattformen und digitaler Angebote.

StudySmarter

Funktionsweise, Zielgruppe, Vorteile/Schwächen, Vergleich mit eigenem Projekt.

1.3.2 Maturatraining digitalisieren

Wie war Maturatraining früher - wie muss digitalisiert werden

1.4 Lernstrategien

In Folge der Entwicklung einer digitalen Lernplattform ist es notwendig, geeignete Lernstrategien zu betrachten. Diese sollen den Lernprozess unterstützen und bestmöglich auf Web und App anwendbar sein. In der Folgenden Analyse werden drei verschiedene Lernstrategien vorgestellt und untersucht. Beachtet werden dabei das Leitner-System, Spaced Repetition und Microlearning.

Evaluationskriterien

- **Fortschrittsvisualisierung:** Hier wird betrachtet, inwiefern die Lernstrategien Daten liefern die gut in Grafiken und Statistiken aufbereitet werden können.
- **Langfristiger Wissenserhalt:** Es wird untersucht, wie gut die Lernstrategien geeignet sind, um Wissen langfristig zu behalten.
- **Plattformunabhängig:** Es wird bewertet, wie gut die Lernstrategien auf verschiedenen Plattformen (Web, Mobile) umgesetzt werden können.
- **Eignung für komplexe Maturathemen:** Es wird analysiert, wie gut die Lernstrategien geeignet sind, um komplexe Themengebiete abzudecken.

1.4.1 Leitner-System

Das Leitner-System stellt eine systematische Lernmethode dar, die auf dem Karteikartenprinzip basiert und vom deutschen Wissenschaftsjournalisten Sebastian Leitner entwickelt wurde. Die Grundidee dieser Methode beruht auf der Erkenntnis, dass Lerninhalte unterschiedlich häufig wiederholt werden sollten, abhängig vom individuellen Beherrschungsgrad des Lernenden.

Funktionsweise

Das System arbeitet mit mehreren Fächern, üblicherweise fünf, in die Lernkarten eingeordnet werden. Jede neue Lernkarte beginnt in Fach 1, welches Inhalte repräsentiert, die noch nicht korrekt beantwortet wurden. Bei richtiger Beantwortung wandert die Karte in das nächsthöhere Fach, bei falscher Beantwortung verbleibt sie im aktuellen Fach oder wird zurück in ein niedrigeres Fach verschoben. Die Wiederholungsfrequenz nimmt mit steigender Fachnummer ab, sodass gut beherrschte Inhalte seltener, schwierige Inhalte hingegen häufiger wiederholt werden.

Effektivität

Die Effektivität des Leitner-Systems basiert auf einer gezielten Ressourcenallokation der Lernzeit. Durch die differentielle Wiederholungsfrequenz wird die verfügbare Lernzeit primär auf jene Inhalte konzentriert, die noch nicht ausreichend verinnerlicht wurden. Dies führt zu einem strukturierten, zeiteffizienteren und zielgerichteteren Lernprozess. Das System eignet sich besonders gut für Faktenwissen und Definitionen, die eine präzise Reproduktion erfordern.

Umsetzung im Projekt

Die digitale Implementierung des Leitner-Systems bietet den Vorteil einer automatisierten Verwaltung der Kartenfächer. Die klare Struktur ermöglicht eine intuitive Bedienung und direkten Überblick über den Lernfortschritt. Allerdings ist die Fortschrittsvisualisierung primär auf die Einordnung in Fächer beschränkt, was für eine detaillierte statistische Auswertung gewisse Limitationen mit sich bringt.

1.4.2 Spaced Repetition

Spaced Repetition, auch als verteiltes Lernen bezeichnet, repräsentiert eine wissenschaftlich fundierte Lernmethode, bei der Lerninhalte in zeitlich optimierten Abständen wiederholt werden. Diese Methode basiert auf der Ebbinghausschen Vergessenskurve und dem Spacing-Effekt, welche zeigen, dass zeitlich verteilte Wiederholungen zu einem nachhaltigeren Wissenserwerb führen als massierte Lerneinheiten.

Funktionsweise

Das Prinzip der Spaced Repetition beruht darauf, dass Lerninhalte kurz vor dem prognostizierten Vergessen erneut präsentiert werden. Das Zeitintervall zwischen den Wiederholungen wird dabei mit jeder erfolgreichen Abfrage progressiv verlängert. Während die erste Wiederholung bereits nach wenigen Stunden erfolgt, können spätere Wiederholungen Wochen oder Monate auseinanderliegen. Dieser Ansatz nutzt die neurophysiologischen Mechanismen des Gedächtnisses optimal aus und führt zu einer nachhaltigen Konsolidierung der Lerninhalte im Langzeitgedächtnis.

Effektivität

Die Effektivität von Spaced Repetition ist durch zahlreiche kognitionspsychologische Studien belegt. Die Methode optimiert den Lernprozess durch die zeitliche Verteilung der Wiederholungen, wodurch das Gelernte signifikant länger im Gedächtnis verankert bleibt. Im Vergleich zu anderen Lernstrategien zeigt Spaced Repetition eine deutlich höhere

Effizienz beim langfristigen Wissenserhalt. Die Methode ist sowohl für kleinere Stoffmengen als auch für umfangreiche Themenkomplexe, wie sie für die Maturavorbereitung relevant sind, geeignet.

Umsetzung digital

Für die digitale Umsetzung bietet Spaced Repetition erhebliche Vorteile. Die automatisierte Berechnung der optimalen Wiederholungszeitpunkte anhand von Algorithmen entlastet den Lernenden von der manuellen Planung. Zudem liefert das System umfangreiche Daten für statistische Auswertungen und Fortschrittsvisualisierungen. Zahlreiche etablierte Lernanwendungen nutzen bereits Spaced-Repetition-Algorithmen, was die praktische Umsetzbarkeit und Akzeptanz dieser Methode unterstreicht. Die Implementierung ist sowohl im Web als auch in mobilen Applikationen gut realisierbar.

1.4.3 Microlearning

Microlearning bezeichnet einen didaktischen Ansatz, bei dem Wissen in kleine, leicht verarbeitbare Lerneinheiten segmentiert wird. Jede Einheit fokussiert auf ein klar abgegrenztes Thema und ist zeitlich auf wenige Minuten begrenzt.

Funktionsweise

Der Lernstoff wird in kurze, modulare Einheiten zerlegt, die verschiedene Medienformate umfassen können: Mini-Videos, kompakte Texte, Quizfragen oder Infografiken. Diese Module können sequenziell oder nach individuellen Präferenzen bearbeitet werden, wobei der Lernprozess primär auf selbstgesteuertem Lernen basiert. Die Flexibilität des Ansatzes ermöglicht Lerneinheiten in verschiedenen Alltagssituationen, beispielsweise während kurzer Pausen oder Pendelzeiten.

Effektivität

Die Effektivität von Microlearning beruht auf der kognitiven Entlastung durch kleine Informationseinheiten, die eine höhere Aufmerksamkeitsspanne und bessere Informationsverarbeitung ermöglichen. Der fokussierte Ansatz, bei dem jeweils nur ein Thema behandelt wird, fördert konzentriertes Lernen. Die zeitliche Kürze der Einheiten erleichtert die Integration in den Alltag und senkt die Hemmschwelle für regelmäßige Lernaktivitäten. Allerdings liegt der Fokus primär auf kurzfristiger Wissensaufnahme, weniger auf langfristiger Retention.

Anwendung

Microlearning eignet sich besonders für Lernanwendungen mit thematisch abgegrenzten Inhalten, die durch kurze Videos oder Texte vermittelt und anschließend durch Quizze gefestigt werden können. Die Methode ist optimal für mobile Lernszenarien konzipiert und lässt sich technisch sehr gut in Web- und App-Umgebungen umsetzen. Für komplexe Maturathemen kann Microlearning eingesetzt werden, sofern diese in mehrere kleinere Einheiten zerlegbar sind, stößt jedoch bei umfangreichen zusammenhängenden Themenkomplexen an Grenzen.

1.4.4 Vergleich verschiedener Lernstrategien und Schlussfolgerung

Kriterium	Microlearning	Leitner-System	Spaced Repetition
Fortschrittsvisualisierung	Prozentanzeige für erledigte Themengebiete	Eingeschränkt, primär Einordnung in Fächer	Optimal für statistische Auswertungen und detaillierte Visualisierungen
Langfristiger Wissensserhalt	Schwerpunkt auf kurzfristiger Wissensaufnahme	Effektiv bei regelmäßiger Anwendung	Sehr effektiv, wissenschaftlich optimierte Wiederholungen
Plattformunabhängigkeit	Sehr gut geeignet, ideal für mobile Nutzung	Umsetzbar, jedoch mit gewissen Einschränkungen	Gut integrierbar, bereits in zahlreichen Anwendungen implementiert
Eignung für Maturathemen	Bedingt geeignet, abhängig von Zerlegbarkeit in kleine Einheiten	Gut für Faktenwissen und Definitionen	Sehr gut, flexibel für verschiedene Stoffmengen und Komplexitätsgrade

Tabelle 1: Vergleichende Bewertung der untersuchten Lernstrategien

Kombination von Leitner-System und Spaced Repetition

Die vergleichende Analyse zeigt, dass jede der untersuchten Lernstrategien spezifische Stärken aufweist. Für die Entwicklung einer digitalen Lernplattform zur Maturavorbereitung wird eine Kombination aus Leitner-System und Spaced Repetition empfohlen, da diese beiden Ansätze sich in ihrer Wirkung komplementär ergänzen.

Das Leitner-System bietet eine klare, intuitiv verständliche Struktur, die sich hervorragend mit digitalen Karteikarten umsetzen lässt. Es ermöglicht einen unmittelbaren Überblick über den Beherrschungsgrad einzelner Lerninhalte und schafft damit eine transparente Lernumgebung. Die Stärke liegt in der einfachen Visualisierung des Lernfortschritts und der eindeutigen Kategorisierung von Lerninhalten.

Spaced Repetition erweitert diesen Ansatz durch die wissenschaftlich fundierte Optimierung der Wiederholungszeitpunkte. Basierend auf kognitionspsychologischen Erkenntnissen über Gedächtnisprozesse und Vergessenskurven werden Wiederholungen automatisch zu den Zeitpunkten eingeplant, an denen die Erinnerung an den Lernstoff zu verblassen droht. Diese Präzision führt zu maximaler Effizienz im Lernprozess.

Die Kombination dieser beiden Systeme bietet folgende Vorteile:

- **Individualisierte Wiederholungsintervalle:** Lerninhalte werden adaptiv an den individuellen Lernfortschritt angepasst, wodurch eine optimale Balance zwischen Wiederholungshäufigkeit und Zeiteffizienz erreicht wird.
- **Transparente Fortschrittsvisualisierung:** Die Struktur des Leitner-Systems ermöglicht eine anschauliche Darstellung des Lernfortschritts, beispielsweise durch digitale Kartenfächer oder grafische Statistiken.
- **Skalierbarkeit:** Das kombinierte System eignet sich sowohl für kleinere Wissensseinheiten als auch für umfangreiche Stoffmengen, wie sie für die Maturavorbereitung charakteristisch sind. Die intelligente Priorisierung von Lerninhalten ermöglicht eine effiziente Bewältigung großer Themenkomplexe.
- **Plattformunabhängige Implementierung:** Die technische Umsetzung ist sowohl in webbasierten Anwendungen als auch in mobilen Apps realisierbar, was eine flexible und ortsunabhängige Nutzung ermöglicht.
- **Alltagstauglichkeit:** Das System lässt sich problemlos in den Schulalltag von HTL-Schülern integrieren und unterstützt sowohl strukturiertes Lernen zu Hause als auch spontane Lerneinheiten unterwegs.

Während das Leitner-System die strukturelle Basis und intuitive Benutzerführung bereitstellt, sorgt Spaced Repetition für die zeitliche Optimierung des Lernprozesses. Diese Synergie resultiert in einem Lernsystem, das sowohl die unmittelbaren Anforderungen der Maturavorbereitung erfüllt als auch langfristigen Wissenserhalt gewährleistet. Die wissenschaftliche Fundierung in Kombination mit praktischer Anwendbarkeit macht diesen hybriden Ansatz zur optimalen Wahl für die geplante digitale Lernplattform.

1.5 Zielsetzung

Was soll am Ende entstehen? Funktionen und Ziele der Plattform.

Das Hauptziel dieser Arbeit ist die Entwicklung einer webbasierten Lernplattform, die Schüler:innen der HTL Leonding dabei unterstützt, sich effizient und gezielt auf die Matura-Prüfung vorzubereiten. Ohne großen Aufwand sollen die User in der Lage sein, individuelle Fragenpools zu erstellen, die ihren persönlichen Lernbedürfnissen entsprechen. Die Plattform soll zusätzlich auf einer effizienten Lernmethode aufbauen, in diesem Fall eine Mischung aus Spaced Repetition und dem Leitner-System, um den Lernerfolg zu maximieren. Durch das visuelle Aufbereiten des Lernfortschritts mittels Diagramme und Statistiken, kann der/die Nutzer:in jederzeit seinen Wissensstand einsehen und gezielt an seinen Schwächen arbeiten. Das gezielte Üben von Schwachstellen wird besonders durch den Einsatz von Lernstrategien und Fehleranalysen unterstützt, die den Wiederholungsbedarf definiert. Um den Wissensstand auf die Probe zu stellen bietet Learnhub eine Prüfungssimulation, diese ebenfalls persönlich vom User angepasst werden kann. Das betrifft sowohl die Auswahl der Themenpools und wie viele Fragen in der Simulation gestellt werden als auch die Zeitbegrenzung für die gesamte Prüfung.

2 Technologische Entscheidungen und Architektur

2.1 Backend

Das Backend fungiert als zentrale Logik- und Datenschicht für beide Teilprojekte der Diplomarbeit. Es stellt die notwendigen Ressourcen für die Verwaltung der Lernmaterialien als auch für das interaktive Lernsystem über eine einheitliche REST-Schnittstelle zur Verfügung.

Dabei muss das Backend folgende projektspezifische Kernaufgaben erfüllen:

- **Zentrale Datenhaltung:** Verwaltung und Persistierung von Fragen, Lösungswegen und Nutzerdaten in einer gemeinsamen Datenbank, um Konsistenz zwischen den Teilprojekten zu gewährleisten.
- **Materialverwaltung:** Abwicklung von Datei-Uploads (Mitschriften, Skripte) und deren strukturierte Zuordnung zu Fachbereichen.
- **Validierung und Business-Logik:** Prüfung von Nutzerantworten im interaktiven Modus und die serverseitige Interpretation von Lernfortschritten.
- **Schnittstellenfunktion:** Bereitstellung von Endpunkten für das Frontend, wobei das Backend die endgültige Entscheidungshoheit über die Datenintegrität besitzt.

Die technologische Wahl ist aufgrund dieser zentralen Rolle also entscheidend für die Performance und Wartbarkeit des Gesamtsystems. Im folgenden Vergleich werden daher die infrage kommenden Frameworks gegenübergestellt.

Analyse technischer Anforderungen

Die Auswahl des Backend-Frameworks ist eine kritische Entscheidung, die Auswirkungen auf die gesamte Lebensdauer der Software hat. Dabei geht es nicht nur um die Syntax, sondern vor allem darum, wie die Anwendung mit der Infrastruktur (Datenbanken, Container) interagiert und wie sie konfiguriert wird.

Folgende Punkte wurden dabei kritisch untersucht:

- **Technische Basis und Ressourceneffizienz:** Untersuchung, wie das Framework intern arbeitet (Optimierung beim Starten vs. Optimierung beim Bauen). Hierbei ist wichtig, wie viel Arbeitsspeicher benötigt wird und wie schnell das Backend in einer Container-Umgebung einsatzbereit ist.
- **Datenbankanbindung und Datenverwaltung:** Analyse, wie einfach sich Datenbanken einbinden lassen und wie gut das Framework dabei hilft, diese als Objekte abzubilden und als solche dann zu persistieren (ORM). Ein wichtiger Punkt ist hier die Flexibilität bei verschiedenen Datenbanksystemen.
- **Einrichtung und automatisierte Abläufe:** Bewertung der Mechanismen für das *Database Seeding*. Es wird geprüft, wie Stammdaten automatisch beim Starten in die Datenbank kommen und wie gut das Framework die nötige Infrastruktur (wie Docker-Container) während der Entwicklung selbst verwaltet.
- **Konfiguration und Sicherheit:** Vergleich, wie Einstellungen über die zentrale Einstellungsdatei gesteuert werden. Dabei geht es vor allem um die Trennung von Entwicklungs- und Live-Umgebungen sowie den sicheren Umgang mit Passwörtern und Zugangsdaten.
- **Zukunftssicherheit und Zusammenarbeit:** Bewertung, wie gut die Dokumentation ist und wie einfach es ist, Hilfe bei Problemen zu finden. Da zwei Teams am selben Backend arbeiten, spielt auch die Erweiterbarkeit für beide Teilprojekte eine große Rolle.

2.1.1 Spring Boot – Die Evolution des Java-Enterprise-Standards



Abbildung 1: Logo von Spring Boot

Historischer Kontext

Spring Boot wurde 2014 veröffentlicht und hatte als Ziel, die massive Komplexität der

damaligen Java-Enterprise-Entwicklung (J2EE/Jakarta EE) zu lösen. Zuvor verbrachten Entwickler einen erheblichen Teil ihrer Zeit mit der Konfiguration von XML-Dateien, wobei man hunderte Zeilen XML-Code schreiben musste, um einfache Datenbankverbindungen zu realisieren. Das wurde schnell zu einer redundanten Arbeit und bekannt als „*Configuration-Hell*“, da man Fehler oft unglaublich schwer finden und ausbessern konnte. Mit Spring Boot mussten die Entwickler:innen nicht mehr wissen, wie ein Webserver startet, sie mussten ihn nur noch benutzen. Es basiert dabei auf dem bewährten Spring Framework und nutzt dessen *Inversion of Control* Container, um die Abhängigkeiten der Anwendung zu verwalten. [1] [2]

Technologischer Durchbruch: Convention over Configuration

Entscheidend für den technologischen Durchbruch war das Prinzip „Convention over Configuration“. Anstatt alles explizit konfigurieren zu müssen, geht Spring Boot dabei einfach davon aus, dass Entwickler in 90% der Fälle dieselben Standardeinstellungen für Datenbanken oder Webserver benötigen. Das Framework erkennt beim Start, welche Bibliotheken im Klassenpfad vorhanden sind und konfiguriert diese automatisch (*Auto-Configuration*). Wenn zum Beispiel eine H2-Datenbank im Klassenpfad liegt, dann möchte der:die Entwickler:in vermutlich eine In-Memory-Datenbank nutzen. Durch diese Automatisierung wurde Java wieder wettbewerbsfähig gegenüber schnellen Frameworks in Ruby- und Python-Umgebungen. [3]

Das führte dazu, dass Unternehmen wie Netflix, Uber oder Zalando Spring Boot als Basis für ihre Microservice-Architekturen wählten.

Ineffizienz in modernen Container-Umgebungen [4]

Obwohl Spring Boot die Entwicklung um ein Vielfaches beschleunigt hat, kommt es in modernen, containerisierten und ressourcenbeschränkten Umgebungen zu einer messbaren Ineffizienz. Das Kernproblem liegt in der hohen Dynamik des Startvorgangs. Während moderne Cloud-Native-Ansätze versuchen, so viel Wissen und Entscheidungen wie möglich in die Kompilierzeit zu verlagern, trifft Spring Boot fast alle architektonischen Entscheidungen erst zur Laufzeit.

Sobald der `main`-Befehl ausgeführt wird, startet das Framework eine Kette von ressourcenintensiven Operationen:

1. Iteratives Classpath Scanning und Bytecode-Analyse:

Da zur Kompilierzeit nicht feststeht, welche Komponenten aktiv sein sollen, muss die Java Virtual Machine beim Start das gesamte Dateiarchiv rekursiv durchsuchen. Spring scannt dabei den rohen Bytecode tausender Klassen nach Annotationen (z. B. `@Service`). Dieser Vorgang skaliert negativ mit der Projektgröße. Je mehr Funktionen die Lernplattform erhält, desto träger und zeitaufwendiger wird dieser Suchvorgang bei jedem Start.

2. Dynamisches Bean-Wiring und Condition-Check:

Nach der Analyse muss Spring Boot entscheiden, welche Komponenten wie miteinander verknüpft werden (Dependency Injection). Dabei werden hunderte Bedingungen geprüft, wie beispielsweise ob eine Datenbank-Library vorhanden ist. Das Framework berechnet erst jetzt einen komplexen Abhängigkeits-Graphen. Ein Konfigurationsfehler wird oft erst am Ende dieser Kette bemerkt, was die Entwicklungszyklen im Vergleich zu proaktiveren Ansätzen verlängert.

3. Runtime Reflection und Proxy-Generierung:

Um essentielle Funktionen wie das Transaktionsmanagement zu ermöglichen, erzeugt Spring Boot zur Laufzeit mittels Reflection künstliche Stellvertreter-Klassen, auch *Proxies* genannt. Das ist nicht nur CPU-intensiv, sondern belegt auch im Arbeitsspeicher permanent Platz. Dies erklärt auch den hohen Memory-Footprint, der bereits im Leerlauf der Anwendung vorhanden ist. (ca. 250-400 MB).

Datenverwaltung und Entwicklerproduktivität im Projektkontext

Für den Erfolg der Lernplattform ist allerdings nicht nur die rein technische Laufzeit-performance ausschlaggebend, sondern auch die Effizienz während der Implementierungsphase. Spring Boot ist Teil des Spring-Ökosystems, und dieses bietet mit *Spring Data JPA* eine der ausgereiftesten Abstraktionen für den Datenbankzugriff. Komplexe Relationen lassen sich durch dieses Object-Relational-Mapping sehr stabil modellieren. Dies stellt einen wesentlichen Vorteil für die Datenintegrität dar, da die Geschäftslogik schon durch die strikte Typisierung von Java geschützt wird, bevor sie überhaupt die Persistenzschicht erreicht.

Trotz dieser Erleichterungen erfolgt bei der praktischen Einrichtung der Infrastruktur ein deutlicher administrativer Aufwand. Dabei folgt Spring Boot einem sehr statischen Konfigurationsansatz über die zentrale Datei `application.properties`. Um das in

einem konkreten Beispiel zu veranschaulichen: Das Framework kann zwar automatisch Objekte mit Tabellen verknüpfen, es kann allerdings nicht die benötigte Infrastruktur wie einen MySQL-Container selbständig verwalten. Daher muss jedes Teammitglied eine passende Datenbankumgebung starten und die Verbindungsparameter exakt denen in der Konfigurationsdatei angleichen.

Natürlich lässt sich der manuelle Setup-Aufwand durch den Einsatz von *Docker-Compose* standardisieren, indem die benötigte Datenbank-Infrastruktur in einer separaten Konfigurationsdatei definiert wird. Während der Entwicklung jedoch müssen die Zugangsdaten und Ports in der `docker-compose.yml` stets manuell mit jenen in `application.properties` synchronisiert werden.

Durch diese Redundanz zwischen Infrastruktur-Definition und Applikations-Konfiguration entsteht eine potenzielle Fehlerquelle. Da zwei Teams gleichzeitig am Code arbeiten, führen Änderungen an der Datenbankstruktur oder den Verbindungsparametern häufig zu „Connection-Refused“-Fehlern bei anderen Teammitgliedern, wenn bei diesen die lokale Umgebung nicht exakt synchronisiert wurde. Spring Boot bietet dafür keine native Integration, die den Lebenszyklus der Docker-Container während der Entwicklung steuert oder auf Strukturänderung im Code reagiert. [2]

Database Seeding via `import.sql` [5]

Ein wesentliches Kriterium für das Entwickeln eines Backends im Zusammenhang mit einer Datenbank ist das *Database Seeding*, also das automatisierte Befüllen der Datenbank mit Stammdaten wie Nutzerdaten und Themenbereiche. Spring Boot bietet hierfür primär den Mechanismus der `import.sql`. Beim Anwendungsstart liest das Framework diese Datei automatisch ein und führt deren SQL-Statements direkt gegen die Datenbank aus, sofern die Property `ddl-auto` auf `create` oder `create-drop` gesetzt ist.

In der praktischen Umsetzung für die Lernplattform offenbaren sich jedoch deutliche konzeptionelle Schwächen:

- **Mangelnde Refactoring-Sicherheit:** Es besteht keine Kopplung zum Java-Quellcode, wodurch eine Umbenennung eines Attributs im Code nicht zu einem Compiler-Fehler im SQL-Skript führt. Der Fehler tritt also erst zur Laufzeit auf, was die Fehlersuche erschweren kann.

- **Eingeschränkte Ausdrucksstärke:** Komplexe Anforderungen, wie das korrekte Hashen von Test-Passwörtern, sind über das SQL-Skript nicht möglich.
- **Kopplung an den Schema-Lebenszyklus:** Die `import.sql` ist strikt an das Neuerstellen der Tabellen gebunden. Das bedeutet, dass bei jeder Änderung an den Testdaten die gesamte Datenbankstruktur gelöscht und neu aufgebaut werden muss. In Kombination mit den bereits beschriebenen langen Startzeiten von Spring Boot führt dies zu einem noch trägereen Entwicklungszyklus, da kein inkrementelles Update der Daten möglich ist.

Fazit

Insgesamt zeigt sich also, dass die Flexibilität von Spring Boot durch einen hohen Preis erkauft wird. Das Framework benötigt einen signifikanten Teil des verfügbaren Arbeitsspeichers und der CPU-Zyklen für die eigene Verwaltung. Auf der oft begrenzten Infrastruktur einer Schule oder in kleinen Cloud-Instanzen wird dieser Overhead zum Problem, da Ressourcen für die Framework-Logik statt für die eigentliche Business-Logik der Lernplattform reserviert werden, wobei es auf solchen Container-Umgebungen wichtig ist, wirklich nur so viele Ressourcen zu verwenden, wie es auch notwendig ist.

Aufgrund dieses Preises für Flexibilität und des tatsächlichen Ressourcenverbrauchs müssen Alternativen evaluiert werden, die entweder ein schlankeres Laufzeitmodell verfolgen oder die Analyseprozesse komplett anders organisieren.

2.1.2 Node.js - Das ereignisgesteuerte Gegenmodell



Abbildung 2: Logo von Node.js

Historischer Kontext

Node.js wurde schon im Jahr 2009 von Ryan Dahl vorgestellt und gilt als Wendepunkt

in der Server-Programmierung. Zu dieser Zeit dominierten Frameworks wie das Spring Framework oder der Apache HTTP Server den Markt. Diese verwendeten das klassische „Thread-per-Request“-Modell. Das heißt für jeden Besucher, der eine Webseite aufrief, wurde ein eigener Thread im Arbeitsspeicher reserviert. Bei einer geringen Nutzerzahl funktionierte dieses Modell auch reibungslos, wenn man aber dann tausende gleichzeitige Verbindungen laufen hatte, verbrauchten diese Systeme den Großteil ihres Arbeitsspeichers nur für die Verwaltung der Threads, anstatt die eigentliche Logik auszuführen. [6]

Dabei erkannte Dahl, dass das Problem nicht bei der CPU lag, sondern bei dem Warten auf Ein- und Ausgabeprozesse wie Datenbankabfragen oder das Lesen von Dateien. Deshalb ersetzte er das blockierende Modell mit einer ereignisgesteuerten Architektur, welche er mit der extrem schnellen V8-JavaScript-Engine von Google Chrome kombinierte. Anstatt also auf eine Antwort der Datenbank zu warten und dabei einen Thread blockieren zu müssen, verwendet Node.js eine *Event-Loop*. Durch diese Architektur war es plötzlich möglich, tausende Verbindungen zu verarbeiten, und dabei nur einen Bruchteil des Arbeitsspeichers zu verwenden, den eine vergleichbare Java-Anwendung benötigt hatte. Das machte JavaScript, das zuvor fast nur im Browser für Animationen genutzt wurde, zu einer wettbewerbsfähigen Technologie für hochperformante Backends. [7]

Risiken des Single Threaded Ansatzes

Die enorme Effizienz von Node.js beruht auf der Annahme, dass der Server hauptsächlich mit „I/O-bound-tasks“, also mit dem Warten auf Lese- und Schreibvorgänge, beschäftigt ist. Da aber die gesamte Anwendung in einem einzigen Thread agiert, ergibt sich im Gegenzug ein Risiko bei rechenintensiven Aufgaben.

Anders als bei Spring Boot, bei welchem rechenintensive Operationen einfach auf einen freien Thread verlagert werden, würde bei Node.js die gesamte Event-Loop einfrieren. Diesen Zustand nennt man „Starvation“ und bei diesem kann der Server keine weiteren Anfragen mehr verarbeiten. Einfachste Funktionen, wie zum Beispiel der Login, blieben in diesem Fall unbeantwortet, bis die CPU-intensive Aufgabe abgeschlossen wurde. [8]

Obwohl Node.js oft als rein „single-threaded“ bezeichnet wird, agiert im Hintergrund eigentlich die C++ Bibliothek *libuv*, welche die zeitintensiven I/O-Operationen in einen internen Worker-Pool auslagert. Das ändert allerdings nichts am fundamentalen Risiko

für die Lernplattform: Da der eigentliche JavaScript-Code trotzdem strikt auf einem einzigen Haupt-Thread läuft, können komplexe Berechnungen innerhalb der Business-Logik nicht automatisch auf diesen Worker-Pool verteilt werden. Dafür benötigt es den expliziten Einsatz von *Worker-Threads* durch die Entwickler:innen, ohne welchen eine hohe CPU-Last im JavaScript-Code zu einer Blockade der Event-Loop führt, da *libuv* nur systemnahe Aufgaben, aber keine benutzerdefinierten Rechenoperationen automatisch parallelisiert. [9] [10]

Für die Lernplattform heißt das, dass eine strikte Trennung zwischen leichtgewichtigen API-Abfragen und schweren Hintergrundprozessen vorherrschen muss. Das wäre zum Beispiel durch Worker-Threads oder durch Microservices möglich, stellt aber auch einen zusätzlichen Architekturaufwand dar. Es ist also notwendig, diesen Aufwand gegen die Vorteile der schnellen Startzeiten gegenüber Spring Boot abzuwägen.

Datenverwaltung und Infrastruktur-Steuerung

Bei der Datenhaltung ist die Philosophie von Node.js in seiner Modularität deutlich erkennbar [11]. Während Java-Ökosysteme durch den JPA-Standard eine formalisierte und herstellerunabhängige Schicht erzwingen, ist der Datenbankzugriff in Node.js geprägt von spezialisierten Bibliotheken und vor allem für JSON-basierte Datenbanken entwickelt. Im Gegensatz dazu gibt es zwar moderne ORMs wie *Prisma* [12] oder *TypeORM* [13], jedoch fehlt diesen die jahrzehntelange Entwicklung und die daraus resultierende Reife und tiefe Integration, die Spring Boot durch die *Auto-Configuration* bietet.

Das bedeutet für die Lernplattform konkret, dass Node.js eine exzellente Anbindung an JSON-basierte Datenbanken, wie zum Beispiel MongoDB, ermöglicht, was für unstrukturierte Lernmaterialien durchaus vorteilhaft sein kann. Auf der anderen Seite erfordert die Nutzung von relationalen SQL-Systemen allerdings eine wesentlich stärkere Absicherung, da Node.js keine native Integration für komplexe Enterprise-Transaktionsmuster bietet.

Automatisierung und Entwicklungszyklus

Für die Effizienz im täglichen Entwicklungsprozesses ist vor allem der Grad der Automatisierung, den die gewählte Technologie bietet, ausschlaggebend. Dabei verfolgt Node.js allerdings einen sehr minimalistischen Ansatz. Während Spring Boot durch seine *Auto-Configuration* versucht, so viele Entscheidungen über die Infrastruktur als

möglich vorab zu treffen, liegt bei Node.js die Verantwortung fast vollständig bei den Entwickler:innen.

Das zeigt sich besonders in der Infrastruktur-Steuerung, wobei keine Mechanismen in Node.js eingebaut sind, um externe Abhängigkeiten, wie Datenbank-Container oder Mock-Services, zu koordinieren. Bei der parallelen Entwicklung von zwei verschiedenen Teams bedeutet das, dass die Synchronisation der lokalen Entwicklungsumgebungen manuell über externe Werkzeuge wie *Docker-Compose* erfolgen muss. Somit erfordert jede Änderung am Datenmodell oder an den Verbindungsparametern eine explizite Kommunikation zwischen den beiden Entwicklungsteams, da das Framework strukturelle Ungleichheiten erst zur Laufzeit bemerkt.

Ein wesentlicher Vorteil aber zeigt sich in einem erheblichen Gewinn an Agilität durch die Art und Weise, wie Node.js auf Code-Änderungen reagiert. Einer der größten Vorteile in der Verwendung dieses Frameworks ist die Schnelligkeit des Feedback-Zyklus. Durch den nativ eingebauten Watch-Modus entfallen die zeitintensiven Startzeiten, die bei Spring Boot durch das Neu-Kompilieren und Scannen des Classpaths entstehen und oft mehrere Sekunden in Anspruch nehmen. Änderungen am Quellcode werden nahezu in Echtzeit in der laufenden Anwendung reflektiert und es ist kein Neustart erforderlich. [14]

Dadurch können also in der Lernplattform iterative Anpassungen an der REST-API wesentlich schneller getestet werden und somit steigt die Effizienz des Entwicklungsprozesses durchaus. Diese Geschwindigkeit kommt allerdings trotzdem mit dem Nachteil, dass Node.js keine native, annotationsbasierte Validierung von Datenmodellen mitbringt. Deshalb müssen Entwickler:innen die repetitive Prüfungslogik manuell implementieren, was das Risiko für Inkonsistenzen in der Geschäftslogik erhöht.

Flexibilität beim Database Seeding

Im Database Seeding zeigt sich ein deutlicher funktionaler Unterschied. Während Spring Boot primär auf SQL-Dateien zum Importieren von Testdaten setzt, die unflexibel gegenüber Änderungen am Datenmodell sind und keine logischen Operationen erlauben, erfolgt das Befüllen der Datenbank in Node.js in der Regel programmatisch. Das hat den Vorteil, dass beispielsweise Passwörter für Test-User direkt während des Seedings gehasht werden können, was im statischen SQL-Ansatz nicht möglich ist. Außerdem nutzen diese Skripte das selbe Datenmodell wie die Anwendung selbst, wodurch strukturelle

Fehler in den Testdaten schon während der Entwicklung und nicht erst zur Laufzeit auffallen. [15]

Konfigurationsmanagement und Umgebungsvariablen

Bei der Konfiguration bietet Node.js kein direktes Äquivalent zur zentral verankerten `application.properties`-Datei von Spring Boot. Stattdessen setzt das Node.js-Ökosystem meist auf Umgebungsvariablen, die häufig über `.env`-Dateien verwaltet werden. [16]

Dieser Ansatz birgt allerdings wesentliche Nachteile im Vergleich zur strukturierten in Java für die Wartbarkeit. Während Spring Boot durch *Configuration Properties* die Werte aus der Einstellungsdatei direkt in typsichere Java-Objekte umwandelt und diese beim Start validiert, werden Umgebungsvariablen in Node.js als einfache Strings eingelesen. Dabei ist Spring Boot „Fail-Fast“, das heißt es stoppt sofort bei Fehlern, und Node.js ist oftmals „Lazy“, das heißt, dass Fehler erst auftreten, wenn der Code-Pfad erreicht wird. Dadurch werden Konfigurationsfehler, wie es zum Beispiel ein fehlender API-Schlüssel oder ein falsch formatierter Port wären, erst spät zur Laufzeit bemerkt und nicht schon beim Start der App abgefangen. Weiters fehlt Node.js ein eingebautes System für Profile wie `dev` oder `prod`. Somit müssen die Teams manuell sicherstellen, dass für jede Umgebung die passende `.env`-Datei geladen wird, was bei der parallelen Arbeit von zwei Teams die Fehleranfälligkeit bei der Bereitstellung der Teilprojekte erhöht.

Fazit

Node.js ist also ein hochperformantes Backend-Framework, allerdings nur spezifische Anwendungsfälle vollständig geeignet. Durch seine ereignisgesteuerte Architektur und die effiziente *Event-Loop* löst es das Ressourcen-Problem von klassischen Java-Anwendungen und bietet zugleich eine Agilität im Entwicklungsprozess, die sich durch nahezu sofortige Feedback-Zyklen auszeichnet. Vor allem beim Database-Seeding übertrifft Node.js durch die Flexibilität von programmatischen Skripten die statischen Ansätze von Spring Boot deutlich.

Dennoch ergeben sich für die Entwicklung der geplanten Lernplattform kritische Nachteile. Die Single-Threaded-Architektur stellt ein wesentliches Risiko für die zukünftige Erweiterbarkeit dar, da rechenintensive Operationen die gesamte Anwendung blockieren könnten. Ein schwerwiegenderes Problem ergibt sich allerdings im Mangel an Standardi-

sierung und nativer Typsicherheit innerhalb des Ökosystems. Da zwei Entwicklerteams an diesem Projekt arbeiten, würde die manuelle Verwaltung von Infrastruktur, Validierung und Konfiguration ein erhöhtes Fehlerrisiko bei der Integration bedeuten.

Somit stehen sich zwei Extreme gegenüber: Auf der einen Seite das robuste, aber ressourcenintensive Spring Boot und auf der anderen Seite das leichtgewichtige, aber architektonisch weniger gefestigte Node.js. Im Zuge dieser Analyse wird also ein weiteres Framework betrachtet. Im bestenfall bringt es die Stabilität, Typsicherheit und die bewährten Standards des Java-Ökosystems, aber auch die Effizienz und moderne Entwickler-Experience von Node.js zusammen. Dieses Anforderungsprofil führt zu **Quarkus**, einem Framework, das explizit als „Supersonic Subatomic Java“ entwickelt wurde, um diese Lücke zu schließen.

2.1.3 Quarkus



Abbildung 3: Logo von Quarkus

Historischer Kontext: Die Java-Krise in der Ära der Containerisierung

Um die Entstehung von Quarkus im Jahr 2019 zu verstehen, muss man zuerst die Geschichte von Java in Server-Umgebungen und den mit den Jahren zum Problem gewordenen Standards kennen. Über Jahrzehnte war Java die unangefochtene erste Wahl für Unternehmenssoftware. Frameworks wie Spring Boot in 2014 perfektionierten die Entwicklung von großen Microservices, die auf leistungsstarken Servern mit gigantischen Arbeitsspeicherkapazitäten liefen. Dabei war Java darauf optimiert, einmal zu starten und über mehrere Wochen hinweg durch den Just-In-Time Compiler [17] im laufenden Betrieb immer mehr an Schnelligkeit zu gewinnen.

Mit der Zeit verschob sich der Standard allerdings immer mehr in Richtung Containerisierung und damit kam der Aufstieg von Docker-Containern und Kubernetes. So änderten sich die Anforderungen fundamental: In modernen Cloud-Umgebungen werden Anwendungen nicht mehr monatelang betrieben, sondern wenn nötig im Bruchteil einer Sekunde hochgefahren und bei Nichtgebrauch sofort wieder abgeschaltet, um Kosten zu sparen. In dieser neuen Welt, in der Minimalverbrauch von Ressourcen eine entscheidende Anforderung wurde, verlor Java zunehmend seinen bisherigen Monopolstatus.

Für viele Expert:innen war dies ein Signal, dass Java nun seinem Ende zuneigte, wenn es um moderne Cloud-Anwendungen geht. Während also Entwicklerteams massenhaft zu Node.js oder Go wechselten, startete aber ein Team bei Red Hat ein ehrgeiziges Projekt mit dem Ziel, Java nicht durch eine neue Sprache zu ersetzen, sondern fundamental so umzubauen, dass es „Cloud-Native“ wird.

Im März 2019 wurde Quarkus als Open-Source-Projekt veröffentlicht. Dabei leitet sich der Name aus der Physik ab, wobei ein Quark der kleinste Baustein der Materie ist und „us“ für die Community steht. Mit dem Slogan „Supersonic Subatomic Java“ war es ihnen also wichtig, einen extrem geringen Speicherverbrauch zu erzielen, der es erlaubt, viel mehr Container auf der selben Hardware zu betreiben als zuvor möglich war, sowie die Startzeiten auf einen wettbewerbsfähigen Stand für moderne Cloud-Architekturen zu reduzieren. [18]

Die technische Revolution: Build-Time Augmentation

Einer der entscheidendsten Faktoren für die Schnelligkeit von Quarkus liegt in der Verschiebung des „Bootstrapping“-Prozesses. Wie bereits bei Spring Boot analysiert, verbringen klassische Frameworks beim Start viel Zeit damit, eine Art „Bestandsaufnahme“ zu machen. Dabei durchsuchen sie tausende Dateien nach Annotationen, bauen komplexe Abhängigkeitsbäume auf und generieren dynamisch Proxies im Arbeitsspeicher. Dieser Prozess findet bei jedem erneuten Starten statt, egal ob auf der Entwickler-Umgebung oder auf dem Cloud-Server.

Im Gegensatz dazu nutzt Quarkus einen Ansatz, den man als „*Shift-Left*“ der Infrastruktur-Logik bezeichnet. Das Framework verlagert also die Intelligenz von der Runtime in die Build-Time. Während also bei Spring Boot beim Start alle Parameter aufs Neue analysiert werden müssen, passiert dies bei Quarkus nur einmal, und zwar beim Kompilieren.[19]

In der Tiefe passiert dabei folgendes:

- **Vorausblickendes Metadaten-Parsing:**

Anstatt zur Laufzeit mühsam nach Annotationen wie `@Inject` oder `@Path` zu suchen, erledigt Quarkus dies einmalig während des Kompilierens. Die Ergebnisse werden in einem effizienten Format gespeichert, das beim Start direkt gelesen werden kann.

- **Vorbereitung der Reflection:**

Reflection, also das dynamische Untersuchen von Klassen zur Laufzeit, ist in Java extrem mächtig, aber auch extrem langsam und speicherintensiv. Quarkus minimiert den Einsatz von Reflection also, indem es den notwendigen Code für die Verknüpfung von Komponenten bereits während des Build-Vorgangs direkt in den Bytecode schreibt.

- **Pre-Bootig der Framework-Logik:**

Viele Framework-Komponenten wie beispielsweise Hibernate werden von Quarkus schon während des Baus „vor-initialisiert“. Alle Entscheidungen, die nicht von Laufzeit-Informationen wie Passwörtern abhängen, sind beim Start der Applikation bereits getroffen.

- **Dead-Code Elimination:**

Quarkus führt beim Bauen eine vollständige Analyse durch. Dadurch kann es präzise bestimmen, welche Teile der eingebunden Bibliotheken niemals aufgerufen werden. Dieser „Dead-Code“ wird entfernt, was die resultierende Datei verkleinert und Speicherbedarf reduziert.

Das Ergebnis dieser „Augmentation“ ist eine Anwendung, die vor dem Start schon fast alle Entscheidungen getroffen hat und somit lediglich den bereits vorbereiteten Plan ausführen muss. Dadurch wurden Startzeiten von unter zwei Sekunden auf der Standard-JVM möglich. [20]

Kritische Betrachtung des Build-Time-Ansatzes

Aus dieser radikalen Verschiebung der Prozesse entsteht allerdings folglich auch ein klassischer technologischer Kompromiss. Da Quarkus den Großteil der Analysearbeit, die normalerweise erst beim Starten der Applikation anfällt, in den Kompilierungsablauf vorzieht, erhöht sich die Build-Zeit auch spürbar. Die Anwendung ist im Betrieb zwar „subatomar“ und schnell, jedoch müssen Entwickler:innen beim finalen Bau des Projekts beispielsweise in einer CI/CD-Pipeline mit längeren Wartezeiten als bei Spring Boot oder Node.js rechnen.

Da Quarkus außerdem beim Bauen bereits den gesamten Code-Pfad kennen muss, um ihn optimieren zu können, ist zum Beispiel das Nachladen von Modulen zur Laufzeit stark eingeschränkt. Außerdem ist das Ökosystem trotz rasantem Wachstum nicht so umfangreich wie das von Spring Boot. Deshalb kann auch nicht jede beliebige

Java-Bibliothek ohne Weiteres effizient genutzt werden, sondern braucht eine spezielle *Quarkus-Extension* [21], die diese für den Build-Time-Prozess vorbereitet. [19]

Für die Entwicklung der Lernplattform bedeutet dieser technologische Kompromiss, dass die Priorität bewusst auf die Stabilität und Effizienz der Laufzeitumgebung gelegt wird. Da das interaktive Lernsystem auf schnelle Antwortzeiten angewiesen ist, um ein flüssiges Lernerlebnis zu gewährleisten, ist die investierte Zeit beim Kompilieren ein vorteilhafter Tausch. Die Einschränkungen bei der dynamischen Modulnachladung sind im Zuge dieser Diplomarbeit vernachlässigbar, da die Architektur der Lernplattform auf einem fest definierten Datenmodell basiert, das von der statischen Analyse von Quarkus profitiert.

Infrastruktur-Automatisierung durch Dev Services

Ein entscheidender Vorteil von Quarkus für die Zusammenarbeit im Team ist die Einführung der sogenannten *Dev Services*. In der vorherigen Analyse von Node.js und Spring Boot wurde die manuelle Koordination von Datenbank-Infrastruktur als potenzielle Fehlerquelle identifiziert, da Zugangsdaten und Container-Zustände zwischen den Teams manuell synchronisiert werden müssen.

Dieses Problem löst Quarkus durch eine tiefe Integration von *Testcontainers*. Wenn das Framework während der Entwicklung feststellt, dass eine Datenbank-Erweiterung vorhanden, aber keine Verbindung konfiguriert ist, so startet es automatisch den entsprechenden Docker-Container im Hintergrund. Dabei übernimmt Quarkus die vollständige Konfiguration der Verbindungsparameter. Das bedeutet für die zwei Teams der Diplomarbeit eine „Zero-Configuration“-Umgebung, also eine Umgebung, in der jedes Teammitglied das Backend ohne vorheriges manuelles Aufsetzen der Datenbank und Abgleichen der Passwörter starten kann. Dies eliminiert die bei Node.js kritisierten „Connection-Refused“-Fehler und sorgt dafür, dass beide Teilprojekte stets auf einer identischen und automatisierten Infrastruktur arbeiten. [22]

Datenbankanbindung und automatisierte Datenverwaltung

Quarkus setzt bei der Handhabung mit Daten auf *Hibernate ORM* mit der möglichen Erweiterung von *Panache*. Während klassische Java-Frameworks oft sehr umfangreichen Code für einfache Datenbankoperationen benötigen, reduziert Panache diesen Aufwand erheblich. [23]

- **Aktive Validierung und Konsistenz:**

Anders als bei Node.js, wo Datenbankänderungen oft erst zur Laufzeit zu Fehlern führen, ist in Quarkus eine strikte Typsicherheit gegeben. Ändert ein Team das Datenmodell, so prüft Quarkus bereits beim Kompilieren, ob die Datenbankstruktur noch zum Java-Code passt. Dies verringert Inkonsistenzen zwischen den Teilprojekten.

- **Automatisiertes Database Seeding:**

Für die Lernplattform ist wichtig, dass nach dem Start sofort Testdaten zur Verfügung stehen. Quarkus unterstützt dies auf zwei Ebenen:

1. **SQL-basiert:** Durch eine einfache `import.sql`-Datei werden beim Start automatisch Tabellen befüllt.
2. **Programmatisch:** Quarkus ermöglicht es, Seeding-Logik direkt in Java zu schreiben. So können beispielsweise verschlüsselte Passwörter für Test-User oder komplexe Lerninhalte dynamisch beim ersten Start generiert werden.

Die zuvor beschriebenen Dev Services gemeinsam mit dieser automatisierten Datenverwaltung schaffen eine „Ready-to-Code“-Umgebung, bei der Entwickler:innen lediglich den Entwicklungsmodus starten und innerhalb von Sekunden eine laufende Datenbank inklusive aller benötigten Testdaten erhält, ohne eine einzige Zeile SQL manuell ausführen zu müssen.

Effizienz im Entwicklungszyklus: Live Coding und Hot Reload

Einer der zentralen Kritikpunkte an klassischen Java-Frameworks wie Spring Boot war in der Vergangenheit der langsame Feedback-Zyklus. Nach fast jeder Code-Änderung musste die Anwendung manuell neu gestartet werden, wodurch häufig Wartezeiten von zehn bis zwanzig Sekunden entstanden.

Quarkus eliminiert diesen Nachteil von Java durch ein hocheffizientes Live Code-System. Wird eine Datei im Quellcode verändert und die Änderung gespeichert, so erkennt Quarkus dies und tauscht nur die betroffenen Klassen im laufenden Betrieb aus. Bei vielen Node.js Konfigurationen wird hierfür der gesamte Server-Prozess neu gestartet. Bei Quarkus allerdings bleibt die JVM stabil, es wird nur die geänderte Geschäftslogik neu in den Speicher geladen. [24]

Dieser Vorgang geschieht in der Regel in weniger als einer Sekunde. Das bedeutet für die Entwicklung der Lernplattform, dass Änderungen an der REST-Schnittstelle oder

der Validierungslogik sofort getestet werden können, ohne dabei den Arbeitsfluss zu unterbrechen.

Zentrale Konfiguration und Sicherheitsaspekte Ein weiterer Faktor, der im Zuge der Analyse behandelt wird, ist die Verwaltung von Umgebungsvariablen und sensiblen Daten. Dabei nutzt Quarkus ähnlich wie Spring Boot eine zentrale Datei namens `application.properties`, die aber eine Besonderheit aufweist: das native Profil-Management. [25]

- **Trennung von Umgebungen:**

Anders als bei Node.js, wo man für unterschiedliche Umgebungen verschiedene Dateien nutzt, erlaubt Quarkus die Definition von Profilen innerhalb einer Datei mit Präfixen (`%dev.db.url` oder `%prod.db.url`). Dadurch wird also niemals versehentlich die Konfiguration für die lokale Entwicklung die Live-Datenbank mit den echten Materialdaten überschreibt.

- **Typsichere Konfiguration:**

Bei Node.js werden Konfigurationen meist als einfache Zeichenketten eingelesen. Da hingegen bietet Quarkus typsichere Konfigurationsobjekte und bricht den Startvorgang sofort mit klaren Fehlermeldungen ab, sollte ein wichtiger Parameter fehlen oder im falschen Format stehen. Somit fährt die Lernplattform nie in einem instabilen Zustand hoch.

- **Sicherheit und Credentials:** Für den Umgang mit Passwörtern und API-Keys unterstützt Quarkus die Einbindung von externen Quellen wie Umgebungsvariablen oder Key-Vaults. Das bedeutet, dass sensible Daten nicht im Quellcode stehen müssen, sondern sicher beim Start der Container-Umgebung injiziert werden können.

Dadurch ergibt sich ein robustes Konfigurationsmodell, das besonders bei der Arbeit in zwei Teams die Fehleranfälligkeit senkt, da die Konfigurationsstruktur durch das Java-Backend fest vorgegeben und validiert wird.

2.1.4 Fazit und finale Technologieentscheidung

Die Analyse verdeutlicht, dass die Wahl des Backend-Frameworks ein Abwägen zwischen Reife, Agilität und Ressourceneffizienz darstellt.

- **Spring Boot** weist zwar eine besondere technologische Reife auf, scheidet aber trotzdem aufgrund des hohen Overheads und der fehlenden nativen Infrastruktur-Automatisierung aus, da dies die parallele Arbeit der beiden Teams unnötig erschweren würde.
- **Node.js** löst zwar das Problem des langsamen Feedback-Zyklus in Spring Boot, birgt jedoch durch die fehlende Striktheit der Standardisierung und nativer Typsicherheit ein zu hohes Risiko für die Datenkonsistenz zwischen den Teilprojekten.
- **Quarkus** erweist sich als die optimale Lösung für die Lernplattform. Es vereint die Typsicherheit und Robustheit des Java-Ökosystems mit einer modernen Entwickler-Experience. Die administrativen Hürden, die bei den anderen Frameworks als kritisch identifiziert wurden, werden durch Features wie *Dev Services* und den *Hot Reload* eliminiert.

Somit ist die Entscheidung nicht nur aufgrund einer performanten Laufzeitumgebung auf Quarkus gefallen, sondern war strategisch für eine effiziente und fehlerresistente Zusammenarbeit im Team.

2.2 Frontend

Frontend Auswahl

Das Frontend ist der Teil der Anwendung, der für die Interaktion mit der Nutzer:innen und dem System dient. Es ist dafür verantwortlich, die bereitgestellten Daten visuell darzustellen, Benutzereingaben zu erfassen und eine reibungslose und ansprechende Nutzererfahrung zu gewährleisten. In dieser Plattform spielt das Frontend die entscheidende Rolle des Vermittlers, indem es die Funktionalitäten des Backends zugänglich macht und den Lernprozess der Nutzer:innen in eine intuitive und benutzerfreundliche Oberfläche übersetzt. Eine wesentliche Aufgabe ist es, die verschiedenen Use Cases, wie den Übungsmodus, Prüfungsmodus und die Fortschrittsübersicht, effektiv umzusetzen und dabei sicherzustellen, dass die Nutzer:innen problemlos durch die Anwendung navigieren können.

Ein weiterer wichtiger Aspekt des Frontends ist die Verwaltung der Benutzerinteraktionen. Dies umfasst das Hochladen von Dateien, das Markieren von Favoriten im Fragenpool, das Absolvieren von Übungen sowie Prüfungen und das Auslösen von Sammel-Operationen zur Organisation der Lernmaterialien. Das Frontend stellt hierbei

sicher, dass Eingaben korrekt erfasst und an das Backend weitergeleitet werden, um eine nahtlose Kommunikation zwischen beiden Komponenten zu gewährleisten.

Durch diese Aufgabenverteilung fungiert das Frontend als die interaktive Ebene, die die abstrakten Daten des Backends für die Nutzer:innen verständlich aufbereitet und nutzbar macht. Als vermittelnde Komponente zwischen systemseitiger Datenverarbeitung und Lernprozess trägt es durch die Gewährleistung einer stabilen und benutzerfreundlichen Bedienung maßgeblich zum individuellen Lernerfolg bei.

Darüber hinaus ist die Responsivität des Frontends von großer Bedeutung. Die Anwendung soll auf verschiedenen Geräten und Bildschirmgrößen konsistent funktionieren, um eine breite Nutzerbasis zu erreichen.

Kriterien für Technologievergleich

Für die Entwicklung des Frontends wurden zwei verschiedene Technologien in Betracht gezogen. In folgenden Abschnitten werden die beiden Frameworks Angular und Flutter vorgestellt und deren Vor- und Nachteile erläutert. Anhand dieser Analyse und speziellen Anforderungen der Lernplattform wurde nach folgenden Kriterien schließlich eine Entscheidung für das am besten geeignete Framework getroffen.

- **Performance und Ladezeiten:** Dieses Kriterium bewertet, wie flüssig und effizient die Anwendung läuft, um eine gute Nutzererfahrung zu gewährleisten.
- **Entwicklungsproduktivität:** Hierbei wird betrachtet, welche Tools und Bibliotheken zur Verfügung stehen, um den Entwicklungsprozess zu beschleunigen und zu vereinfachen.
- **Zustandsmanagement:** Dies beschreibt die Fähigkeit des Frameworks, den Zustand der Anwendung zu verwalten und zu synchronisieren, insbesondere bei komplexen Interaktionen und Datenflüssen.
- **Community-Support und Dokumentation:** Ein große Community und gute Dokumentation sind entscheidend, um bei Problemen schnell Lösungen zu finden und bessere Methoden zu nutzen.
- **Typensicherheit und Wartbarkeit des Codes** Typensicherheit hilft, Fehler frühzeitig zu erkennen und verbessert die Wartbarkeit des Codes, was besonders bei größeren Projekten wichtig ist.

Angesichts dieser vielfältigen Aufgabenbereiche wird deutlich, wie entscheidend die Wahl einer geeigneten Frontend-Technologie für die Usability und Performance der

Plattform ist. Im folgenden Abschnitt werden daher zwei verschiedene Frameworks analysiert und im Hinblick auf die spezifischen Anforderungen dieser Lernplattform bewertet.

2.2.1 Angular

Was ist Angular. Vor- und Nachteile usw

Angular ist ein weit verbreitetes Open-Source-Framework für die Entwicklung von Webanwendungen, das von Google entwickelt und gepflegt wird. Es basiert auf TypeScript, einer von Microsoft entwickelten Programmiersprache, die JavaScript erweitert und Typensicherheit bietet. Das Framework ist besonders für die Erstellung von Single-Page-Anwendungen (SPAs) geeignet, bei denen die gesamte Anwendung auf einer einzigen Webseite geladen wird und dynamisch Inhalte aktualisiert werden können, ohne die Seite neu zu laden. Angular verfolgt das Model-View-ViewModel (MVVM)-Architekturmuster, das die Trennung von Logik, Darstellung und Datenmodell fördert und somit die Wartbarkeit und Skalierbarkeit von Anwendungen verbessert.



Abbildung 4: Logo von Angular

Architekturprinzip

Angular zeichnet sich durch seine komponentenbasierte Architektur aus, die es Entwicklern ermöglicht, wiederverwendbare UI-Komponenten zu erstellen und zu verwalten, was schnelle Entwicklung und saubere Wartbarkeit ermöglicht. Das Framework bietet eine Vielzahl von integrierten Funktionen, darunter Datenbindung, Dependency Injection, Routing und Formulare, die die Entwicklung komplexer Anwendungen erleichtern. Ein wesentlicher technisches Merkmal von Angular ist die Verwendung von Dependency Injection. Hierbei werden Abhängigkeiten, wie beispielsweise Services, automatisch zur Laufzeit verwaltet und bereitgestellt. Dies fördert lose Kopplung und erleichtert das Testen und die Wartung von Anwendungen.

Angular Material

Angular Material ist die offizielle UI-Komponentenbibliothek für Angular, die von Google entwickelt wurde. Sie basiert auf den Prinzipien des Material Designs, einem von Google entwickelten Designsystem. Es zielt darauf ab, eine konsistente und funktionelle Benutzeroberfläche zu schaffen und es somit den Entwicklern zu erleichtern. Angular Material bietet eine Vielzahl von vorgefertigten UI-Komponenten, die speziell für Angular-Anwendungen entwickelt wurden. Diese Komponenten sind vollständig anpassbar und können problemlos in Angular-Projekte integriert werden.



Abbildung 5: Logo von Angular Material

Technische Besonderheiten

- **RxJS (Reactive Extensions):** Ein Herzstück von Angular ist die reaktive Programmierung mit RxJS, die es ermöglicht, asynchrone Datenströme zu verwalten und komplexe Ereignisverarbeitungen effizient zu gestalten. Observables und Operatoren bieten leistungsstarke Werkzeuge zur Handhabung der Daten.
- **Angular CLI:** Das Angular Command Line Interface (CLI) ist ein mächtiges Werkzeug, das den Entwicklungsprozess erheblich vereinfacht. Es ermöglicht das schnelle Erstellen, Testen und Bereitstellen von Angular-Anwendungen durch eine Vielzahl von Befehlen.
- **Starke Typensicherheit:** Durch die Verwendung von TypeScript bietet Angular eine starke Typensicherheit, die hilft, Fehler frühzeitig zu erkennen und die Codequalität zu verbessern.
- **Ahead-of-Time Compilation (AOT):** Angular nutzt Ahead-of-Time Compilation, bei der der Code vor der Ausführung also bereits während des Build-Prozesses optimiert wird. Dies verbessert die Ladezeiten und Performance der Anwendung, da dieser Schritt des Interpretierens für den Browser der Nutzer:innen entfällt. Ebenfalls können dadurch früher Fehler erkannt werden.

Vor-und Nachteile

Vorteile	Nachteile
Vollständiges Framework: Bietet integrierte Lösungen für Routing, Formulare und HTTP-Anfragen aus einer Hand.	Steile Lernkurve: Erfordert eine intensive Einarbeitung in Konzepte wie RxJS und Dependency Injection.
Strikte Architektur: Erzwingt eine komponentenorientierte Struktur, die auch bei großen Projekten für hohe Wartbarkeit sorgt.	Bundle-Größe: Die umfangreiche Bibliothek führt zu größeren Bundle-Größen, was die Ladezeiten beeinträchtigen kann.
Hohe Produktivität: Das Angular CLI automatisiert repetitive Aufgaben und beschleunigt den Entwicklungsprozess.	Hoher Overhead: Für sehr einfache, statische Funktionen wirkt die Struktur oft überdimensioniert und überflüssig.
Typensicherheit: Durch TypeScript werden Fehler bereits zur Entwicklungszeit erkannt und minimiert.	Komplexität: Die strikte vorgegebene Architektur kann zu Einschränkungen im individuellen Gestaltungsfreiheit führen.

Tabelle 2: Gegenüberstellung der Vor- und Nachteile von Angular

2.2.2 Flutter

Was ist Flutter. Vor- und Nachteile usw

Flutter ein Open-Source-UI-Toolkit, das von Google entwickelt wurde und es ermöglicht, nativ kompilierte Anwendungen für mobile, Web- und Desktop-Plattformen zu erstellen. Ursprünglich für die mobile Entwicklung (iOS und Android) konzipiert, hat sich Flutter zu einem vielseitigen Framework entwickelt, das eine breite Palette von Plattformen unterstützt, wie auch Webanwendungen und Desktop-Anwendungen für alle Betriebssysteme. Flutter verwendet die Programmiersprache Dart, die ebenfalls von Google entwickelt wurde und sich durch ihre einfache Syntax und hohe Performance auszeichnet. Der technische Kern von Flutter basiert auf einer eigenen Rendering-Engine, die es ermöglicht, benutzerdefinierte UI-Komponenten zu erstellen und eine konsistente Darstellung über verschiedene Plattformen hinweg zu gewährleisten. Im Gegensatz zu anderen Frameworks, die auf HTML-Elemente und CSS des Browsers basieren, zeichnet Flutter oft direkt auf ein eigenes Feld (Canvas).



Abbildung 6: Logo von Flutter

Architekturprinzip

Die Oberfläche von Flutter-Anwendungen weist einen komponentenbasierten Aufbau auf. Bei diesem werden Widgets als Bausteine für die Benutzeroberfläche verwendet. Diese Widgets können sowohl einfache UI-Elemente wie Buttons und Textfelder als auch komplexe Layouts und Animationen umfassen. Flutter bietet eine Vielzahl von vorgefertigten Widgets, die anpassbar sind und die Entwicklung wesentlich erleichtern. Ein zentrales Merkmal von Flutter ist das Hot-Reload-Feature, das es Entwicklern ermöglicht, Änderungen am Code in Echtzeit zu sehen, ohne die Anwendung neu starten zu müssen. Zur Trennung von Logik und Darstellung wird in Flutter häufig das BLoC (Business Logic Component)-Muster verwendet, das eine klare Struktur für die Verwaltung des Anwendungszustands bietet. Dabei werden Events an den BLoC gesendet, diese werden verarbeitet und der aktualisierte State wird an die UI zurückgegeben. Der Provider ist währenddessen eine weitere beliebte Methode zur Zustandsverwaltung in Flutter-Anwendungen. Er ermöglicht die einfache Übermittlung von Daten und Informationen bereitstellt und die abhängigen Widgets automatisch aktualisiert, wenn sich der Zustand ändert.

Technische Besonderheiten

- **Widgets:** Widgets sind die grundlegenden Bausteine von Flutter-Anwendungen. Sie sind unveränderlich (immutable) und beschreiben das Design und das Verhalten der Benutzeroberfläche.
- **Zustandsmanagement(State Management):** Flutter bietet verschiedene Mechanismen zur Zustandsverwaltung, darunter den Provider, BLoC und Redux. Diese ermöglichen eine effiziente und skalierbare Verwaltung des Anwendungszustands.

- **Rendering Engine:** Flutter verwendet eine eigene Rendering-Engine, die es ermöglicht, benutzerdefinierte UI-Komponenten zu erstellen und eine konsistente Darstellung über verschiedene Plattformen hinweg zu gewährleisten.
- **Single Codebase:** Flutter ermöglicht es, mit einem einzigen Codebase Anwendungen für iOS, Android und Web zu erstellen. Das reduziert den Entwicklungsaufwand und gleichzeitig vereinfacht es die Wartung der Anwendung.

Vorteile	Nachteile
Plattformunabhängigkeit: Ermöglicht die Entwicklung für Web, Mobile und Desktop aus einer einzigen Codebasis.	Web-Performance: Da die gesamte Engine geladen werden muss, sind initiale Ladezeiten im Browser oft höher.
Schnelle UI-Entwicklung: Dank umfangreicher Widget-Bibliotheken lassen sich komplexe und interaktive Oberflächen zügig umsetzen.	SEO und Indexierung: Da die Inhalte oft auf ein "S Zeichenfeld" (Canvas) gemalt werden, können Suchmaschinen wie Google den Text der Seite schlechter lesen. Für öffentliche Webseiten ist das ein großer Nachteil.
Hot Reload: Code-Änderungen werden in Millisekunden sichtbar, ohne den aktuellen Anwendungszustand zu verlieren.	Eingeschränkte Web-Integration: Der Zugriff auf spezifische Browser-APIs oder bestehende JavaScript-Bibliotheken ist oft umständlicher.
Hohe Design-Konsistenz: Die eigene Rendering-Engine garantiert ein identisches Aussehen der Anwendung auf allen Endgeräten.	Ökosystem-Fokus: Viele Plugins sind primär für mobile Endgeräte optimiert und bieten im Web-Kontext teilweise weniger Stabilität.

Tabelle 3: Gegenüberstellung der Vor- und Nachteile von Flutter

2.2.3 Ergebnis

Warum wir uns für Angular entschieden haben

Für die Entwicklung des richtigen Frontends ist die Wahl des passenden Frameworks von entscheidender Bedeutung. Das passende Framework bildet den Grundstein für eine gute Benutzerfahrung, eine effiziente Entwicklung und eine langfristige Wartbarkeit der Anwendung. In der folgende Abschnitten werden die beiden Frameworks Angular und Flutter anhand bestimmter Kriterien bewertet und miteinander verglichen.

Kriterienbewertung: Angular

- **Performance und Ladezeiten:** Angular bietet eine gute Performance für SPAs (Single-Page-Application) im Web die aus einer einzigen HTML Seite besteht.

Insbesondere durch die Nutzung von Ahead-of-Time (AOT) Compilation, die den Code vor der Ausführung optimiert.

- **Entwicklungsproduktivität:** Mit dem Angular CLI und einer Vielzahl von integrierten Tools ermöglicht Angular eine hohe Entwicklungsproduktivität.
- **Zustandsmanagement:** Angular bietet verschiedene Möglichkeiten für das Zustandsmanagement, darunter Services und Bibliotheken wie NgRx, die eine effiziente Verwaltung des Anwendungszustands ermöglichen.
- **Community-Support und Dokumentation:** Angular verfügt über eine große und aktive Community sowie umfangreiche offizielle Dokumentation, die Entwicklern bei der Lösung von Problemen und der Implementierung neuer Funktionen hilft.
- **Typensicherheit und Wartbarkeit des Codes** Die Verwendung von TypeScript in Angular fördert die Typensicherheit und verbessert die Wartbarkeit des Codes erheblich.

Kriterienbewertung: Flutter

- **Performance und Ladezeiten:** Flutter bietet eine solide Performance für Webanwendungen, jedoch können die initialen Ladezeiten aufgrund der Notwendigkeit, die gesamte Flutter-Engine zu laden, höher sein. Aufgrunddessen das Canvas Kit beim ersten Aufruf geladen werden muss. Danach läuft Flutter jedoch extrem flüssig.
- **Entwicklungsproduktivität:** Flutter ermöglicht eine schnelle UI-Entwicklung, insbesondere durch das Hot-Reload-Feature, das Änderungen am Code in Echtzeit sichtbar macht. Zudem beschleunigt die umfangreichen vorgefertigten Widgets die Entwicklungszeit.
- **Zustandsmanagement:** Flutter bietet verschiedene Mechanismen zur Zustandsverwaltung, darunter den Provider und BLoC, die eine effiziente Verwaltung des Anwendungszustands ermöglichen. Dadurch ist Flutter flexibler jedoch auch uneinheitlicher.
- **Community-Support und Dokumentation:** Flutter hat eine wachsende Community und eine gute Dokumentation, vorallem bei der Mobile-Entwicklung, bei Web-spezifischen Problemen jedoch noch nicht so umfangreich.

- **Typensicherheit und Wartbarkeit des Codes** Die Verwendung von Dart in Flutter bietet Typensicherheit, jedoch ist die Wartbarkeit des Codes stark von der gewählten Architektur und den verwendeten Zustandsmanagement-Methoden abhängig.

Direkter Vergleich

Kriterium	Bewertung Angular	Bewertung Flutter
Performance und Ladezeiten	Gut	Befriedigend
Entwicklungsproduktivität	Sehr gut	Sehr gut
Zustandsmanagement	Sehr gut	Gut
Community-Support und Dokumentation	Sehr gut	Gut
Typensicherheit und Wartbarkeit	Sehr gut	Gut

Tabelle 4: Direkter Vergleich der Kriterienbewertung zwischen Angular und Flutter

Fazit

Auf Basis der durchgeführten Kriterienbewertung und der Anforderungen der beiden Frameworks wurde Angular ausgewählt. Dadurch ist sichergestellt, dass die Lernplattform durch den komponentenbasierten Aufbau, die starke Typensicherheit und die umfangreichen integrierten Funktionen von Angular eine hohe Wartbarkeit, Skalierbarkeit und Performance bietet. Während Flutter eine interessante Alternative darstellen würde, besonders aufgrund der Plattformübergreifenden Möglichkeiten. Angular überzeugte letztendlich mit der besseren Ladezeit und problemlosen Zusammenarbeit mit Quarkus im Backend.

3 Praktische Umsetzung

3.1 Logische Programmstruktur und Use Cases

Der User Flow (Abbildung 7) visualisiert den typischen Interaktionsablauf der Schüler und Schülerinnen mit der Lernplattform. Dabei ist wichtig, welche Schritte der User von der Anmeldung bis zur Lernaktivität durchläuft und welche Handlungsmöglichkeiten ihm dabei zur Verfügung stehen.

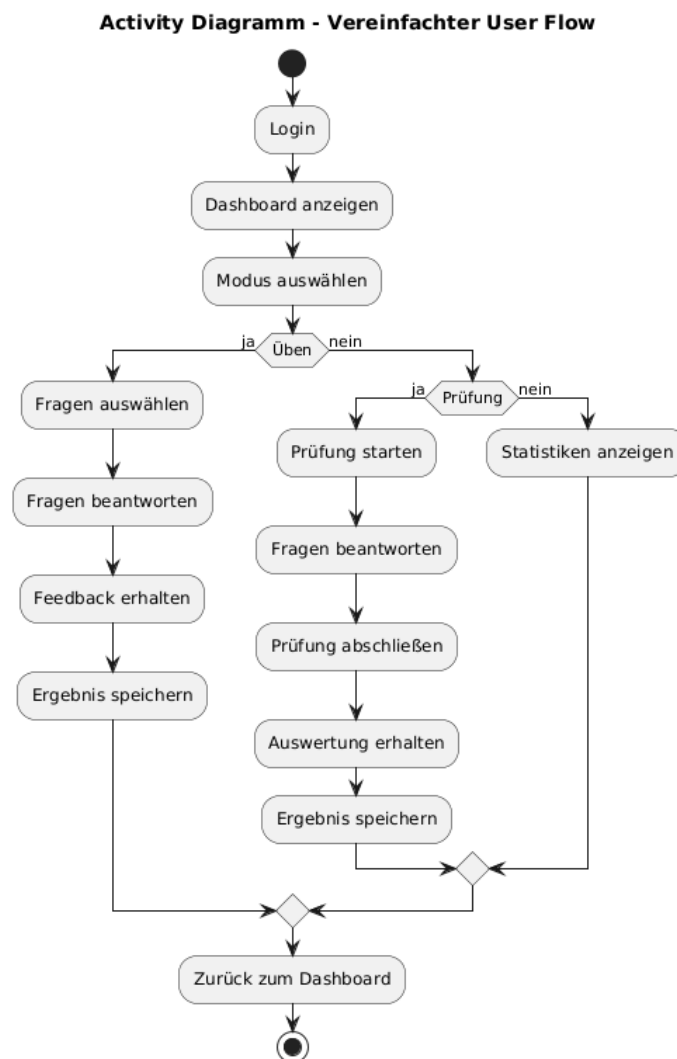


Abbildung 7: Ablaufdiagramm eines Users
[26]

Aus diesem User Flow lassen sich 3 zentrale Abläufe erkennen, die den Kernfunktionen der Lernplattform entsprechen:

- **Übungsmodus:**

Der Übungsmodus beginnt beim gezielten Stöbern in Themenbereichen und der Aufnahme von Fragen in einen persönlichen Fragenpool für systemgestützte Wiederholungen. Wichtig dabei ist der Lernprozess durch unmittelbares Feedback nach jeder Antwort, was die in Kapitel 1.4 untersuchte Lernstrategie durch die Persistierung des Wissenstands effektiv umsetzt.

- **Prüfungssimulationen:**

Bei den Prüfungssimulationen geht es um die Leistungsüberprüfung unter realitätsnahen Bedingungen, bei welchem der aktuelle Wissenstand auf die Probe gestellt werden kann. Nutzer und Nutzerinnen können hierbei frei die zu überprüfenden Themenbereiche, Fragenanzahl sowie das Zeitlimit konfigurieren. Im Gegensatz zum Übungsmodus erfolgt die Auswertung aber nicht nach jeder Antwort, sondern erst nach finaler Abgabe, um authentische Prüfungssimulationen zu gewährleisten.

- **Fortschrittsübersicht:**

In der Fortschrittsübersicht steht die Transparenz und Motivationssteigerung im Vordergrund. Dies wird durch die Visualisierung von täglichen Streaks, den Übungsstatus spezifischer Themenbereiche sowie den chronologischen Prüfungsverlauf erzielt. Dadurch wird eine objektive Einschätzung der Prüfungsreife ermöglicht und eine langfristige Lernbereitschaft durch die grafische Aufbereitung gefördert.

3.2 Use Case: Üben

Der Kern der Anwendung ist der Übungsmodus, welcher Schüler:innen ermöglicht, gezielt Maturafragen zu trainieren und ihr Wissen Stück für Stück auszubauen. Anders als beim Prüfungsmodus steht hier nicht der Leistungsdruck im Vordergrund, sondern das aktive Lernen mit direktem Feedback. Nutzer:innen soll es ermöglicht werden, ein Fundament für ihr Wissen festigen zu können.

Funktionen umfassen das Auswählen von bestimmten Themen, das Zusammenstellen von einem eigenen Fragenpool und das Beantworten von Fragen ohne Stress. Nutzer:innen können hierbei also genau die Themen lernen, die ihnen Probleme bereiten, und das durch eigene Fragen oder Fragen anderer Nutzer:innen.

Die Plattform unterstützt dabei verschiedene Fragetypen, um das Lernerlebnis flexibler zu gestalten. Um nicht einfach nur ein Ergebnis von „richtig“ und „falsch“ zu liefern, können Nutzer:innen auch Lösungswege einsehen, um das Verständnis nachhaltig zu fördern.

Durch diese wiederholbare und flexible Übungsumgebung eignet sich die Anwendung sowohl für den schnellen Gebrauch als Wissenscheck als auch für die langfristige Vorbereitung auf Tests, Schularbeiten oder die Matura.

3.2.1 Verwaltung von Fragenpools

Die Verwaltung von Fragenpools ist eine zentrale Funktion der Lernplattform, die es Nutzer:innen ermöglicht, ihre Lerninhalte individuell zu gestalten und gezielt zu üben. Hier spielt die „Fragen browsen“-Ansicht, wie in Abbildung 8 gezeigt, eine wichtige Rolle, da diese den Nutzer:innen erlaubt, ihre individuell selektierten Fragen gesammelt zu verwalten, und somit genügend Raum für gezieltes Lernen bietet.

Meinen Fragenpool üben > Fragen browsen

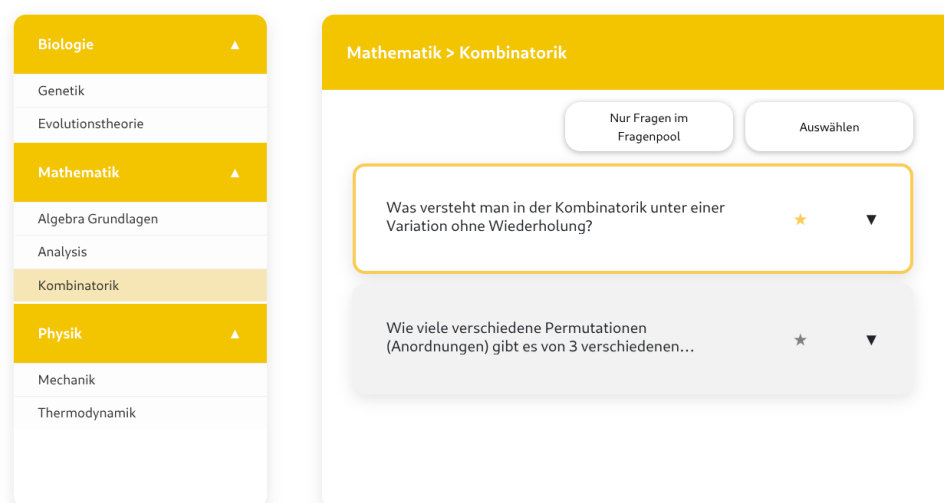


Abbildung 8: Ansicht der Verwaltung des Fragenpools

Das Interface der Fragenpool-Verwaltung ist so gestaltet, dass Nutzer:innen schnell und intuitiv auf ihre gespeicherten Fragen zugreifen können. Die Ansicht ist in zwei Bereiche unterteilt: Links befindet sich eine Seitenleiste, die als Navigation zwischen den Fragen dient, während der Hauptbereich die passenden Fragen zum gewählten Thema anzeigt.

Im Hauptbereich kann zwischen „bereits in Fragenpool und alle“ gewählt werden, wobei gefiltert wird, welche Fragen angezeigt werden sollen. Die Fragen können jederzeit hinzugefügt oder entfernt werden, indem entsprechende Buttons wie alle auswählen oder das Sternsymbol neben der einzelnen Frage verwendet werden.

Jede Frage wird in einer kompakten Kartenansicht dargestellt, die wesentliche Informationen wie die Fragestellung, dazugehörige Bilder und die Möglichkeit enthält, genau diese Frage direkt zu üben.

Datenfluss in der Fragenpool-Verwaltung

Der zentrale Teil der Fragenpool-Verwaltung ist die Interaktion mit dem Backend, um Fragen zu speichern, abzurufen und nach Fächern und Themen zu filtern.

Navigation und Themenkatalog

Sobald der/die Nutzer:in die Fragenpool-Verwaltung öffnet, sendet das Frontend eine Anfrage an das Backend, um den Gesamtkatalog der verfügbaren Fächer und Themen abzurufen. Diese Fächer und Themen werden in der Seitenleiste angezeigt, sodass der/die Nutzer:in schnell zwischen verschiedenen Themen wechseln kann, die dann parallel in der Hauptansicht dargestellt werden.

Die Struktur der Daten ist hierarchisch aufgebaut: Ein Subject (Fach) enthält mehrere TopicPools (Themenpools), die wiederum verschiedene Questions (Fragen) beinhalten. Diese Hierarchie wird im Frontend in verschachtelten Dropdowns dargestellt, wobei der Zustand jedes Dropdowns (geöffnet/geschlossen) in einem Objekt gespeichert wird, das die jeweiligen IDs als Schlüssel verwendet.

Fragen abrufen und Anzeige in der Hauptansicht

Beim Laden der Fragenpool-Verwaltung sendet das Frontend eine Anfrage an das Backend, um die Liste der Fragen abzurufen, die der/die Nutzer:in zuvor in seinen/ihren Fragenpool aufgenommen hat. Diese Fragen werden in einem lokalen Array gespeichert und dienen somit als Referenz für die Anzeige der bereits ausgewählten Fragen. Die Fragen, die schon im Fragenpool sind, werden visuell mittels ausgefülltem Sternsymbol hervorgehoben (siehe Abbildung 8).

Zusätzlich kann der/die Nutzer:in zwischen zwei Modi wählen:

- **Browse-Modus:** Zeigt alle verfügbaren Fragen im gesamten Katalog an.

- **Pool-Modus:** Zeigt nur die Fragen, die bereits im eigenen Fragenpool enthalten sind.

Das Backend antwortet mit einer strukturierten Liste von Frageobjekten, die alle notwendigen Informationen für die Anzeige im Frontend enthalten. Die Komponente verwendet eine berechnete Getter-Methode, die abhängig vom aktuellen Modus entweder die direkten Fragen (Browse-Modus) oder die aus den QuestionPoolEntry-Objekten extrahierten Fragen (Pool-Modus) zurückgibt. Diese Architektur ermöglicht es, dieselbe Komponente für beide Ansichten wiederzuverwenden.

Hinzufügen und Entfernen von Fragen

Wenn der/die Nutzer:in eine Frage zu seinem/ihrer persönlichen Pool hinzufügen möchte, wird durch das Klicken auf das Sternsymbol oder den Alle auswählenButton eine Anfrage an das Backend gesendet. Das Backend speichert die Zuordnung zwischen Nutzer:in und Frage in der Datenbank und bestätigt die erfolgreiche Operation.

Die Implementierung dieser Funktionalität folgt dem Prinzip des Optimistic UI Updates: Sobald der/die Nutzer:in auf das Sternsymbol klickt, wird die Anzeige sofort aktualisiert, ohne auf die Antwort des Backends zu warten. Die Frage-ID wird unmittelbar zum Array der ausgewählten Fragen hinzugefügt, wodurch das Sternsymbol seinen Zustand ändert. Parallel dazu läuft die Backend-Anfrage im Hintergrund. Diese Implementierung sorgt für eine flüssige Benutzererfahrung, da keine Ladezeiten wahrgenommen werden.

Umgekehrt kann eine Frage durch erneutes Klicken auf das Sternsymbol aus dem Pool entfernt werden. Auch hier erfolgt eine sofortige Aktualisierung der UI, während die Backend-Anfrage asynchron verarbeitet wird. Das Backend löscht die entsprechende Zuordnung aus der Datenbank. Die Änderungen werden sofort im Frontend reflektiert, sodass die Nutzer:innen unmittelbares visuelles Feedback erhalten.

Für die Mehrfachauswahl existiert ein spezieller Auswahlmodus, der über einen Button aktiviert werden kann. In diesem Modus ändert sich das Verhalten der Fragenkarten: Statt die Details einer Frage anzuzeigen, werden Fragen zur Auswahlliste hinzugefügt oder entfernt. Dies ermöglicht es, mehrere Fragen gleichzeitig auszuwählen und sie dann mit einer einzigen Backend-Anfrage zum Pool hinzuzufügen (Batch-Operation). Diese Funktionalität ist besonders nützlich, wenn Nutzer:innen ihren Pool initial mit vielen Fragen befüllen möchten.

Filterung und Suche

Die Fragenpool-Verwaltung unterstützt verschiedene Filteroptionen, um die Übersichtlichkeit bei großen Fragenkatalogen zu gewährleisten:

- **Filterung nach Fach:** Über die Seitenleiste können Nutzer:innen gezielt Fragen eines bestimmten Faches anzeigen lassen.
- **Filterung nach Thema:** Innerhalb eines Faches kann weiter nach spezifischen Themengebieten gefiltert werden.
- **Pool-Status:** Die Ansicht kann auf bereits ausgewählte Fragen beschränkt oder auf alle verfügbaren Fragen erweitert werden.

Die Filterung nach Pool-Status erfolgt clientseitig ohne erneute Backend-Anfragen. Dafür wird eine Kopie aller geladenen Fragen in einem separaten Array gespeichert. Beim Aktivieren des Filters werden nur jene Fragen angezeigt, deren IDs im Array der ausgewählten Fragen enthalten sind. Beim Deaktivieren wird die vollständige Liste wiederhergestellt. Diese Implementierung minimiert die Netzwerklast und ermöglicht sofortige Filteränderungen ohne Wartezeit.

Diese Filtermöglichkeiten erleichtern es den Nutzer:innen, gezielt die Fragen zu finden und zu üben, die für ihre aktuelle Lernphase relevant sind.

Technische Umsetzung

Frontend-Architektur

Die zentrale Ansicht für den Fragenpool wird von der `QuestionBrowsingViewComponent` gesteuert. Sie hält den aktuellen Zustand der Ansicht und greift über Services auf das Backend zu. Diese Services kümmern sich um die Kommunikation mit dem Server und liefern sauber strukturierte Daten zurück.

Der `QuestionPoolService` übernimmt dabei die wichtigsten Aufgaben rund um den Fragenpool: Er lädt alle Fragen eines Nutzers, zeigt Fragen zu bestimmten Themen an, fügt neue Fragen hinzu, entfernt bestehende und aktualisiert die Statistik.

Backend-Architektur

Wenn Fragen zum Fragenpool hinzugefügt werden, lädt das Backend zuerst den bestehenden Pool des Nutzers. Anschließend wird geprüft, ob eine Frage bereits vorhanden ist, um doppelte Einträge zu vermeiden. Nur neue Fragen werden dem Pool hinzugefügt. Die Verbindung zwischen Fragenpool und Frage erfolgt über ein eigenes Objekt, das

zusätzliche Informationen speichert. Dazu gehört, wie oft eine Frage richtig beantwortet wurde und ob sie zuletzt korrekt gelöst wurde. Diese Daten sind später wichtig für Statistiken und ein angepasstes Lernverhalten. Alle Änderungen an der Datenbank laufen abgesichert ab: Entweder wird alles korrekt gespeichert oder bei einem Fehler vollständig zurückgesetzt. So bleiben die Daten immer konsistent. Die Struktur der Fächer und Themen ist klar hierarchisch aufgebaut: Ein Fach enthält mehrere Themenpools, und jeder Themenpool besteht aus mehreren Fragen. Diese Struktur wird direkt in der Benutzeroberfläche dargestellt und erleichtert die Navigation.

3.2.2 Fragenhandling und Fragetypen

Wie in Abbildung 9 dargestellt befindet sich der/die Nutzer:in nun beim Question-Runner, also der zentralen Ansicht, bei der Fragen beantwortet und ausgewertet werden können. Diese Komponente ist eine dynamische Ansicht, da sie je nach gewähltem Modus (Üben, Prüfen oder Review) verschiedene Steuerungselemente bereitstellt.

Benutzeroberfläche des Question Runners

Üben > Fragen beantworten

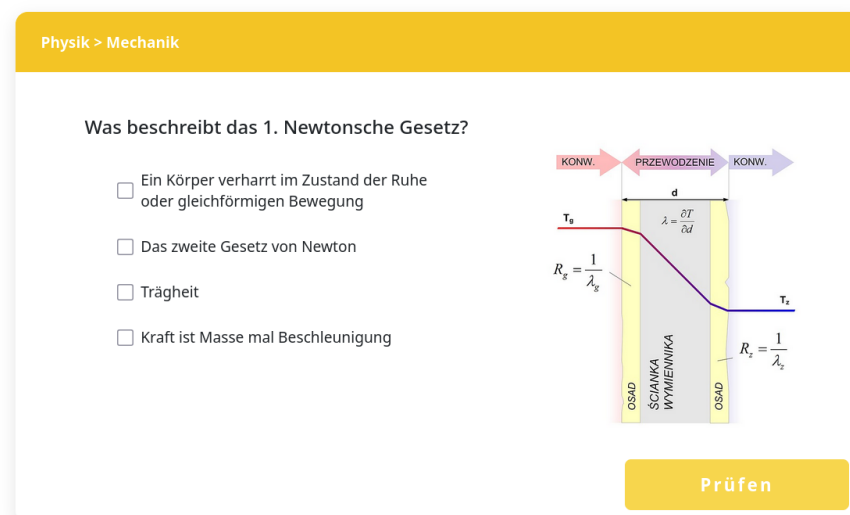


Abbildung 9: Beispielhafte Ansicht einer Frage im Question Runner

Die Ansicht ist hierarchisch aufgebaut, um für eine schnelle Orientierung zu sorgen:

- **Informationsbereich:**

Am oberen Rand wird das aktuelle Fach sowie der spezifische Themenpool angezeigt.

- **Inhaltsbereich:**

Die Fragestellung wird auffällig als Überschrift präsentiert und bei Bedarf auf der rechten Seite durch eine illustrative Grafik ergänzt.

- **Interaktionsbereich:**

Wie nachfolgend beschrieben erscheinen hier je nach Fragetyp Auswahlfelder (Multiple-Choice) oder ein Texteingabefeld (Freitext).

- **Aktionsleiste:**

Im unteren Bereich befinden sich die Navigationselemente. Im Übungsmodus ist dies primär der Button „Prüfen“ zur sofortigen Auswertung, während in der Prüfungssimulation (Kapitel 3.3.2) Navigationsbuttons zum Vor- und Zurückblättern dominieren.

Unterstützung verschiedener Fragetypen

Neben der Unterscheidung der Modi werden im Question Runner die verschiedenen Fragetypen berücksichtigt. Aktuell werden zwei Haupttypen unterstützt:

- **Multiple-Choice-Fragen:** Nutzer:innen wählen eine oder mehrere richtige Antworten aus einer Liste vorgegebener Auswahlmöglichkeiten aus. Multiple-Choice-Fragen sind besonders geeignet, um Faktenwissen abzufragen oder schnelle Wissenskontrollen durchzuführen. Das System wertet die Antwort automatisch aus und kann im Übungsmodus direkt Feedback geben. Die Darstellung erfolgt als übersichtliche Auswahlfelder, die klar zwischen ausgewählt und nicht ausgewählt unterscheiden. (siehe Abbildung 10a)
- **Freitext-Fragen:** Nutzer:innen geben ihre Antwort in ein Texteingabefeld ein. Dieser Fragetyp eignet sich für offene Aufgabenstellungen, bei denen die Antwort nicht auf eine vorgegebene Auswahl beschränkt ist. Für die Eingabe wird ein größeres Eingabefeld bereitgestellt, um längere Antworten komfortabel angeben zu können. Die Eingabe wird mit einer Menge an korrekten Antworten abgeglichen. Entspricht die Eingabe einer dieser Antwortmöglichkeiten, wird die Antwort als korrekt bewertet, andernfalls als falsch. (siehe Abbildung 10b).

Üben > Fragen beantworten

Physik > Mechanik

Was beschreibt das 1. Newtonsche Gesetz?

- ☐ Ein Körper verharrt im Zustand der Ruhe oder gleichförmigen Bewegung
- ☐ Das zweite Gesetz von Newton
- ☐ Trägheit
- ☐ Kraft ist Masse mal Beschleunigung

Prüfen

(a) Multiple-Choice-Frage

Üben > Fragen beantworten

Mathematik > Algebra Grundlagen

Was ist $2 + 2$?

Bitte Antwort eingeben...

Prüfen

(b) Freitext-Frage

Abbildung 10: Question-Runner: Beispiele für verschiedene Fragetypen

Durch diese Gestaltung unterstützt der Question Runner den/die Nutzer:in auf effektive Weise beim Lernen, beim Prüfen des eigenen Wissens und beim Überprüfen bereits bearbeiteter Aufgaben. Die modulare Darstellung der Fragen, der direkte Zugriff auf Feedback im Übungsmodus und die klare Trennung der Modi tragen dazu bei, dass sich der/die Nutzer:in jederzeit auf das Lernen konzentrieren kann, ohne von technischen Details abgelenkt zu werden.

3.2.3 Übungsmodus mit direktem Feedback

Beim Aufruf des Übungsmodus landet der User zunächst auf einer Übersichtsseite, auf der eine Zusammenfassung des aktuellen Lernstands zu finden ist und die einen Einstieg in den nächsten Übungsdurchlauf ermöglicht. Diese Ansicht dient dazu, einen schnellen Überblick darüber zu geben, wie gut einzelne Fragen bereits beherrscht werden und welche Bereiche noch wiederholt werden sollten.

Üben

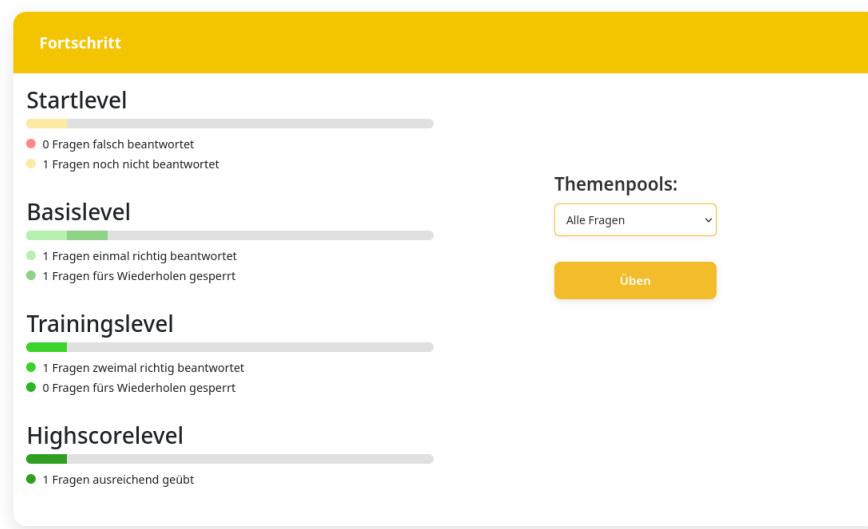


Abbildung 11: Fortschrittsübersicht des Übungsmodus

Auf dieser Übersichtsseite werden alle Fragen der ausgewählten Themenbereiche nach ihrem bisherigen Bearbeitungsstatus gruppiert. Dazu zählen unter anderem:

- (gelb) Fragen, die noch nie beantwortet wurden
- (rot) Fragen, die zuletzt falsch beantwortet wurden
- (hellgrün) Fragen, die bereits 1× korrekt beantwortet wurden
zum Wiederholen gesperrt oder nicht gesperrt
- (grün) Fragen, die bereits 2× korrekt beantwortet wurden
zum Wiederholen gesperrt oder nicht gesperrt
- (dunkelgrün) Fragen, die ausreichend häufig richtig beantwortet wurden

Diese Einteilung hilft dem:der Nutzer:in dabei, den eigenen Lernfortschritt einzuschätzen und gezielt zu entscheiden, welche Fragen noch verstärktere Wiederholung benötigen. Gleichzeitig hebt sie hervor, welche Inhalte sicher beherrscht werden und daher im Übungsmodus seltener erscheinen. Diese Übersicht macht außerdem sichtbar, welche Fragen noch nicht wiederholt werden sollten, da sie erst vor kurzer Zeit geübt wurden. Das sind jene Fragen, die als *"fürs Wiederholen gesperrt"* markiert werden.

Rechts neben der Statusübersicht kann der Themenbereich gewählt werden, auf den sich sowohl die Fortschrittsübersicht als auch die nächste Übungssitzung beschränken soll. So kann sich der Übungsmodus auf bestimmte Fächer oder Themenbereiche eingrenzen

lassen. Wird aber bei dieser Auswahl *Alle Fragen* angegeben, so wird der Übungsmodus auch auf alle Themenbereiche im eigenen Fragenpool ausgeweitet.

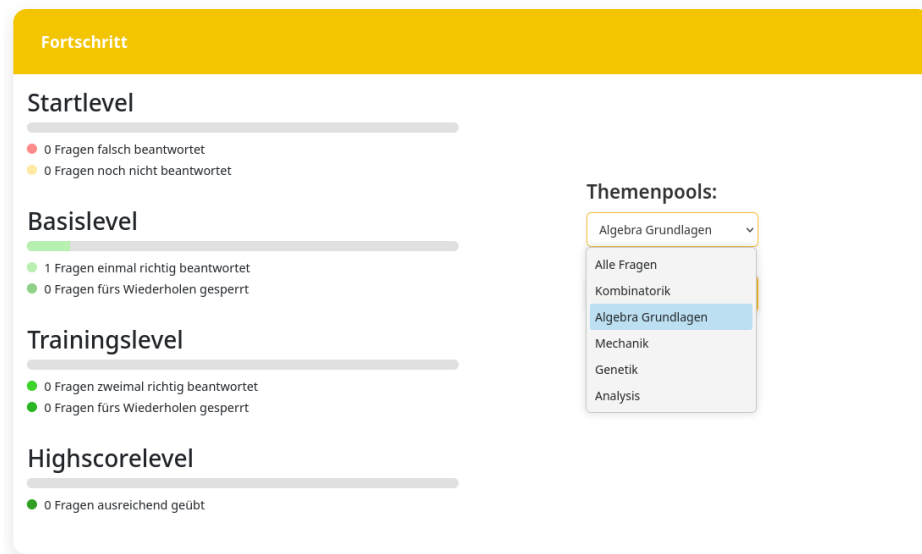


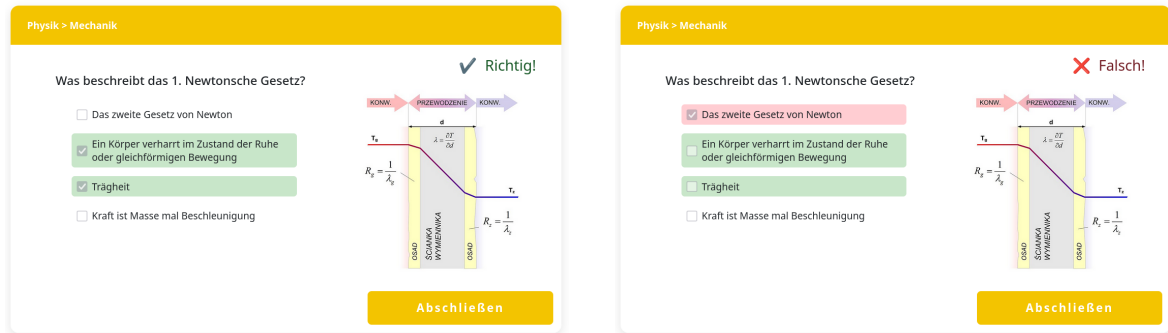
Abbildung 12: Auswahl des Themenbereichs für den nächsten Übungsdurchlauf

Nachdem der gewünschte Themenbereich ausgewählt wurde, kann der Übungsdurchlauf über den "ÜbenButton direkt unter der Auswahl des Themenbereichs gestartet werden. Das System wechselt automatisch in den Question Runner und lädt die nächste passende Frage aus dem ausgewählten Themenbereich, je nachdem bei welchem Fragenstatus aktuell am meisten Übungsbedarf besteht. Auf diese Weise kann die Übungssitzung direkt an dem Punkt starten, an dem es zu diesem Zeitpunkt am wichtigsten ist.

Bearbeitung der Fragen und direktes Feedback

Beim Start in den Übungsdurchlauf gelangt der/die Nutzer:in in den Question Runner, in dem die Fragen nacheinander bearbeitet werden. In diesem Modus liegt der Fokus auf dem Erlernen der ausgewählten Inhalte. Es ist dabei also wichtig, dass es keine Elemente gibt, die möglicherweise Stress oder Ablenkung verursachen, wie sie in klassischen Prüfungssituationen häufig auftreten. Deshalb ist diese Ansicht auch besonders schlicht und simpel gehalten. Der:Die Nutzer:in steht hier nicht unter Zeitdruck und kann die Frage in Ruhe lesen, überdenken und schließlich beantworten. Erst nachdem eine Antwort ausgewählt beziehungsweise eingegeben wurde, kann diese über den "Prüfen"-Button bestätigt werden. Anschließend wird die Antwort ausgewertet und das Ergebnis unmittelbar im Question Runner angezeigt.

Das direkte Feedback ist so gestaltet, dass es den Lernprozess aktiv unterstützt. Nach der Auswertung wird die Frage deshalb visuell markiert und Antworten werden entsprechend farbig markiert. Korrekte Antworten werden mit einem positiven Grünton, falsche Antworten rot hervorgehoben (siehe Abbildung 13). Zusätzlich wird die richtige Lösung und ein möglicher Lösungsweg (siehe Kapitel 3.2.4) angezeigt. So können Fehler sofort nachvollzogen und Missverständnisse direkt geklärt werden.



(a) Korrekt beantwortete Frage

(b) Falsch beantwortete Frage

Abbildung 13: Question-Runner: Richtig und Falsch beantwortete Frage

Durch diese Form der Rückmeldung wird das Gelernte mit der jeweils bearbeiteten Frage verknüpft. So bleibt der Lernprozess nicht abstrakt, sondern findet direkt am Beispiel der Aufgabe statt. Zugleich ermöglicht die strukturierte Darstellung, dass auch falsch beantwortete Fragen nicht nur negativ und als Fehler wahrgenommen, sondern auch als positiver Bestandteil des Lernprozesses genutzt werden.

Nachdem die Auswertung erfolgt ist, wechselt der "Prüfen"-Button zum "Nächste Frage"-Button. Dieser Button bringt den/die Nutzer:in direkt zur nächsten passenden Frage des ausgewählten Themenbereichs, wobei die Auswahl der nächsten Frage der in Kapitel 1.4 beschriebenen Lernstrategie folgt. Der Übungsmodus führt die Nutzer:innen somit schrittweise durch die Inhalte, die für den weiteren Lernfortschritt am relevantesten sind. Dabei muss der/die Nutzer:in nicht selbstständig die nächste Frage auswählen, sondern kann sich einfach vom System leiten lassen.

Wiederholungslogik und erneute Präsentation von Fragen

Nach der Bearbeitung einer Frage wird deren Ergebnis gespeichert. Richtig beantwortete Fragen gelten zunächst als *zum Wiederholen gesperrt* und sind erst nach einer definierten Wartezeit wieder im Übungsmodus vorzufinden. So wird vermieden, dass dieselbe Frage unmittelbar hintereinander erneut gestellt wird, sondern erst zu einem späteren Zeitpunkt, wenn der Lerneffekt größer ist. Falsch beantwortete Fragen bleiben im

Übungsmodus, bis sie korrekt beantwortet wurden, sodass der/die Nutzer:in direkt das Verständnis der gemachten Fehler überprüfen kann. Das soll verhindern, dass Fehler ein weiteres Mal gemacht werden und es zu einem endlosen Kreislauf wird, da man den Fehler nie wirklich verstehen konnte.

Fragen, die mehrfach korrekt beantwortet wurden, werden nach und nach als gefestigt eingestuft. Wurde eine Frage bereits dreimal hintereinander richtig beantwortet, so gilt sie als ausgelernt und wird dauerhaft aus dem aktiven Fragenpool für den Übungsmodus entfernt. Sie erscheint also nicht mehr im Übungsmodus, da davon ausgegangen werden kann, dass der Inhalt ausreichend gefestigt ist.

Verhalten bei Sitzungsfortsetzung und Abschluss des Übungsdurchlaufs

Das explizite Abbrechen des Übungsdurchlaufs ist nicht vorgesehen, verlässt der/die Nutzer:in den Question Runner während der Bearbeitung dennoch, geht der aktuelle Bearbeitungsstand verloren, die bereits beantworteten Fragen bleiben aber gespeichert. Deshalb kehrt der/die Nutzer:in beim erneuten Aufrufen des Übungsmodus also auch automatisch zur zuletzt offenen Frage zurück.

Sind für den ausgewählten Themenbereich vorübergehend keine weiteren Fragen mehr verfügbar, weil alle Fragen entweder ausgelernt oder zur Wiederholung gesperrt sind, so wird der "Prüfen-Button zum Abschließen"-Button. Durch diesen gelangt man zurück zur vorherigen Ansicht, also der Fortschrittsübersicht.

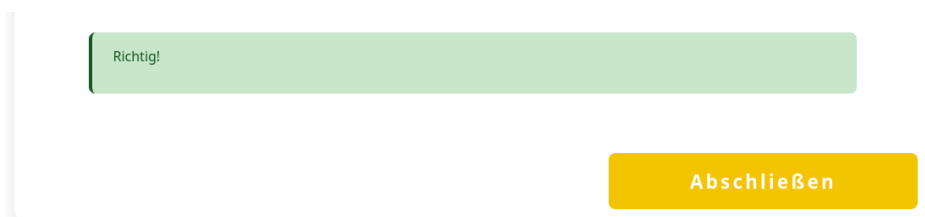


Abbildung 14: "PrüfenButton wurde zum AbschließenButton

3.2.4 Lösungswege und Nutzerinteraktion

Einsehen von Lösungswegen, Feedbackmechanismen

3.3 Use Case: Prüfungssimulation

Im Prüfungsmodus können Nutzer:innen ihr Wissen, das sie im Übungsmodus (3.2.3) erlangt haben, auf die Probe stellen. Er richtet sich also an Schüler:innen, die bereits gelernt und ihr Wissen gefestigt haben, und nun eine Prüfungssituation simulieren möchten, um ihren aktuellen Wissensstand einzuschätzen.

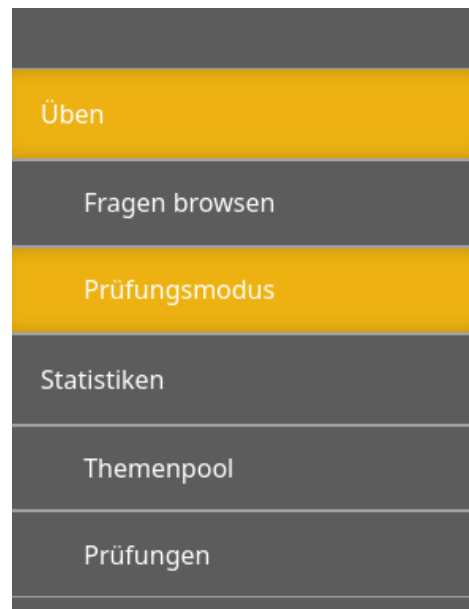


Abbildung 15: Seitennavigationsleiste bei ausgewähltem Prüfungsmodus

Wie in der obenstehenden Abbildung erkenntlich, ist der Prüfungsmodus über die Seitennavigation der Lernplattform zugänglich. Er dient ausschließlich zur Überprüfung des Wissensstands, nicht aber des aktiven Lernens. Die Kernfunktionen umfassen die Bearbeitung einer Reihe an Fragen aus ausgewählten Themenbereichen unter Zeitlimit, die Navigation zwischen Fragen während der Prüfung sowie die Auswertung der Prüfung am Ende mit einer Übersicht über richtig und falsch beantworteter Fragen.

Nutzer:innen können realistische Prüfungssituationen erleben, ohne dass dies Auswirkungen auf ihren Fortschritt im Übungsmodus hat. So bleibt Üben und Prüfen klar voneinander getrennt, während gleichzeitig ein Eindruck über den eigenen Lernfortschritt gewonnen wird. Dabei kann auch ein Einblick über die eigene Performance unter Zeitdruck helfen, neue Baustellen aufzudecken.

3.3.1 Konfiguration einer Prüfung

Die erste Ansicht, wenn der/die Nutzer:in zum Prüfungsmodus navigiert, ist die Konfigurationsansicht. Vor dem Start der Prüfung kann der/die Nutzer:in die Prüfung

individuell konfigurieren. Die Optionen werden dafür übersichtlich auf der Ansicht dargestellt, sodass der/die Nutzer:in gezielt entscheiden kann, welche Themenbereich getestet werden sollen, wie viele Fragen die Prüfung beinhalten soll und welches Zeitlimit dafür zur Verfügung stehen soll.

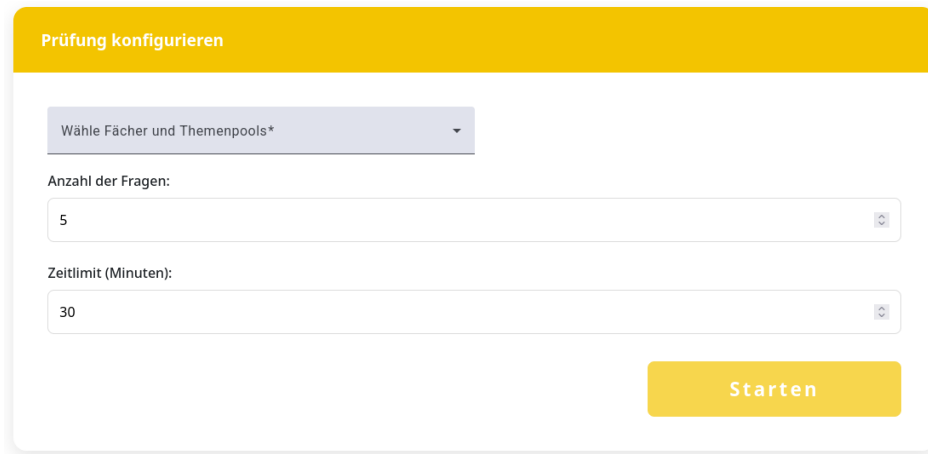


Abbildung 16: Beispielhafte Ansicht der Prüfungs-Konfiguration

Folgende Optionen sind dabei vorhanden:

- **Themenbereich auswählen:**

Nutzer:innen können einzelne Fächer und Themenbereiche auswählen. Dabei werden nur jene ausgewählt, von welchen Fragen auch im eigenen Fragenpool vorhanden sind.

- **Anzahl der Fragen:**

Es kann festgelegt werden, wie viele Fragen insgesamt in der Prüfung enthalten sein sollen. Dies ermöglicht sowohl kurze Tests als auch längere Prüfungssimulationen. Die Prüfung besteht dann aus zufälligen Fragen aus dem Fragenpool über die ausgewählten Bereiche, wobei zu jedem Themenbereich mindestens eine Frage gestellt wird, wenn die Anzahl der Fragen dafür ausreicht.

- **Zeitlimit:**

Das gewünschte Zeitlimit für die gesamte Prüfung kann eingestellt werden, um die Situation einer realen Prüfung nachzubilden. Läuft die Zeit aus, so wird die Prüfung automatisch mit den aktuell eingegebenen Antworten abgeschickt und ausgewertet.

Sobald die Konfiguration abgeschlossen ist, kann der/die Nutzer:in die Prüfung über den *Starten*-Button beginnen. Die Plattform wechselt automatisch in den Prüfungs-

modus im Question Runner, wobei die zuvor festgelegten Konfigurationen angewendet werden.

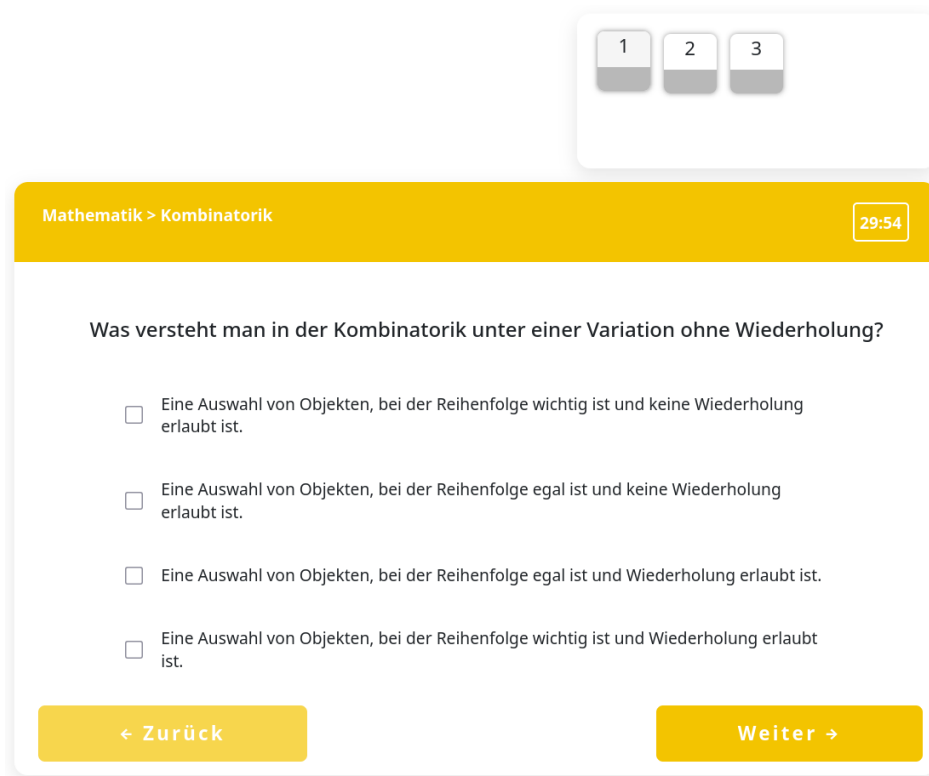


Abbildung 17: Start der Prüfung nach Konfiguration

3.3.2 Prüfungsmodus mit Zeitlimit

Ist die Prüfung gestartet, so gelangt der/die Nutzer:in in den Prüfungsmodus des Question Runners. Dieser Modus soll möglichst authentisch eine realistische Prüfungssituation nachbilden und sich somit klar vom Übungsmodus unterscheiden. Der Fokus soll hierbei nicht mehr auf dem Lernen liegen, sondern auf der Überprüfung des aktuellen Leistungsstands unter Zeitdruck.

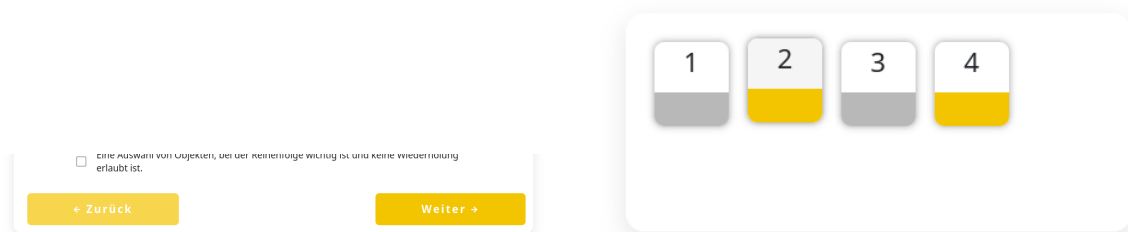
Wie in der oberen Abbildung 17 dargestellt wird in der oberen Zeile nun zusätzlich zum Fach und Themenpool wie im Übungsmodus auch permanent die verbleibende Zeit angezeigt. Durch diese Darstellung soll der zeitliche Rahmen der Prüfung verdeutlicht werden, ohne dabei zu stark abzulenken. Die aktuelle Frage wird darunter angezeigt inklusive Antwortfeldern und gegebenenfalls das zugehörige Bild, analog zur Darstellung im Übungsmodus.

Der zentrale Unterschied zum Übungsmodus ist das verzögerte Feedback. Im Gegensatz zum Übungsmodus erhält der/die Nutzer:in während der Prüfung keinerlei Rückmeldung darüber, ob eine Frage korrekt oder falsch beantwortet wurde. Es kann lediglich zwischen

den einzelnen Fragen beliebig oft hin und her navigiert werden, während sich das System die eingegebenen Antworten merkt. Dadurch kommt der Prüfungsmodus einer realen Prüfung besonders nahe, da man bis zum Ende der Prüfung seine Antworten beliebig oft verändern kann, doch nur die letzte wirklich zählt.

Navigation durch die Prüfung Der/Die Nutzer:in kann während der Prüfung beliebig oft zwischen Fragen hin und her navigieren und dabei seine Antworten beliebig oft ändern. Das Navigieren erfolgt über 2 Arten:

- ***SZurück*"- und "Weiter"-Buttons** (siehe Abbildung 18a)
Ermöglichen es, zur nächsten oder zur letzten Frage zu wechseln.
- **Hauptnavigationsleiste über dem Question Runner** (siehe Abbildung 18b)
Ermöglicht es, zu einer beliebigen Frage in der Prüfung zu springen.



(a) Weiter- und Zurück-Buttons im Prüfungsmodus

(b) Hauptnavigationsleiste des Prüfungsmodus

Abbildung 18: Prüfungsmodus: Navigationsarten

Die *SZurück*"- und *Weiter*"-Buttons navigieren den/die Nutzer:in zur vorherigen beziehungsweise nächsten Frage. Befindet sich der/die Nutzer:in bereits auf der ersten Frage der Prüfung, so ist der *SZurück*"-Button ausgegraut und es ist nicht möglich, eine weitere Frage zurück zu springen. Erreicht der/die Nutzer:in allerdings die letzte Frage, so wird der *Weiter*"-Button zum *Fertigstellen*"-Button, mit welchem die Prüfung abgegeben werden kann.



Abbildung 19: Prüfungsmodus: Letzte Frage - Fertigstellen-Button

Alternativ zu den beiden Navigations-Buttons gibt es auch die Prüfungsnavigationsleiste über dem Question Runner. Diese ermöglicht es, jederzeit zu jeder beliebigen Frage zu

springen. Wie in Abbildung 18b zu erkennen, repräsentiert jeder dieser Boxen eine eigene Frage, welche entweder grau oder orange markiert ist. Diese Farben haben folgende Bedeutungen:

- **Grau:** Diese Frage wurde noch nicht beantwortet
- **Orange:** Für diese Frage wurde bereits eine Antwort gespeichert. Diese Markierung gibt allerdings keine Auskunft darüber, ob die abgegebene Antwort korrekt oder falsch ist.

Abgabe der Prüfung Läuft das vorgegebene Zeitlimit ab, so wird die Prüfung automatisch abgegeben. In diesem Fall werden alle bis dahin gespeicherten Antworten zur als richtig oder falsch ausgewertet. Alle Fragen, die bis zum Ablauf der Zeit nicht beantwortet wurden, gelten dementsprechend auch als unbeantwortet und werden automatisch als falsch gewertet. Der/Die Nutzer:in kann die Prüfung somit nicht über die konfigurierte Dauer hinaus fortsetzen.

Neben der automatischen Abgabe durch Zeitablauf kann die Prüfung auch jederzeit manuell abgeschlossen werden. Wie im oberen Abschnitt bereits angeführt, erfolgt dies über den *"Fertigstellen"*-Button, der beim Erreichen der letzten Frage angezeigt wird. Durch das manuelle Abgeben wird die Prüfung sofort beendet und alle gespeicherten Antworten werden zur Auswertung weitergegeben. Auch hier ist eine nachträgliche Änderung der Antworten ab diesem Zeitpunkt nicht mehr möglich.

Während der gesamten Prüfung werden die eingegebenen Antworten regelmäßig zwischengespeichert, was das Erhalten des Bearbeitungsstands auch beim Navigieren ermöglicht. Ein explizites Pausieren oder Abbrechen der Prüfung ist allerdings nicht vorgesehen. Wird die Prüfung vorzeitig verlassen, so gilt die Prüfung als nicht ordnungsgemäß abgeschlossen und kann nicht fortgesetzt werden. Dies sorgt für einen authentischeren Umgang mit Prüfungssituationen, welche nicht einfach abgebrochen und neu gestartet werden können.

3.3.3 Bewertung und Ergebnisausgabe

Nach dem Abschluss einer Prüfung durch Zeitablauf oder manueller Abgabe wird die Prüfung automatisch ausgewertet. Der/Die Nutzer:in gelangt im Anschluss zur Ergebnisansicht, in der eine zusammenfassende Auswertung der eben abgelegten Prüfung dargestellt wird.

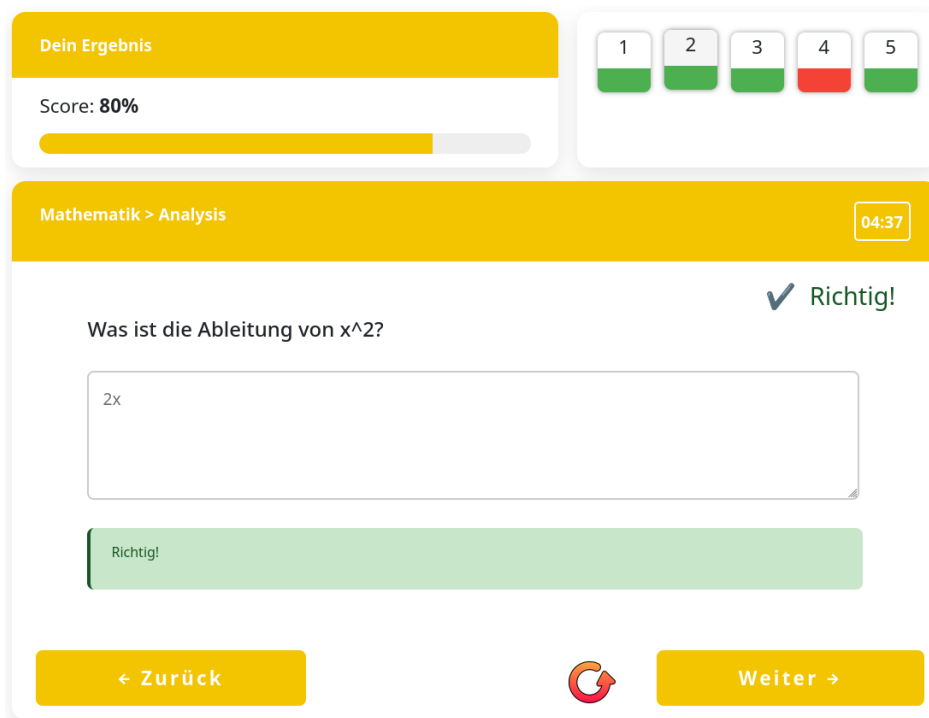


Abbildung 20: Ergebnisübersicht nach Abschluss der Prüfung

In dieser Übersicht kann auf den ersten Blick erkannt werden, wie viele Fragen korrekt beziehungsweise falsch ausgewertet wurden. Zusätzlich wird der erreichte Anteil korrekt beantworteter Fragen in Prozent dargestellt. Auf Basis dieser Anzeige kann der/die Nutzer:in direkt grob einschätzen, wie sehr die ausgewählten Themenbereiche unter Zeitdruck beherrscht werden.

Neben der Gesamtauswertung bietet die Ergebnisansicht auch einen detaillierten Überblick über alle einzelnen Fragen der Prüfung. Die Navigation zwischen den Fragen erfolgt hierbei analog zum Prüfungsmodus. Es ist also sowohl über die "Zurück"- und "Weiter"-Buttons als auch über die Navigationsleiste über dem Question Runner möglich, schnell zwischen Fragen zu springen. Die Navigationsleiste unterscheidet sich allerdings darin, dass bei dieser sofort sichtbar ist, ob die jeweilige Frage korrekt oder falsch beantwortet wurde. Dafür ist die Box für die zugehörige Frage grün beziehungsweise rot markiert.

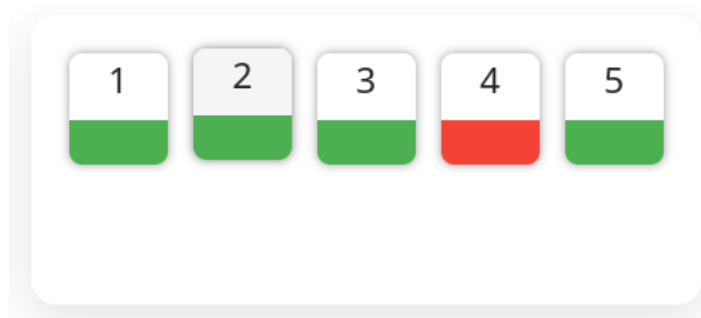


Abbildung 21: Navigationsleiste nach Auswertung der Prüfung

Beim Aufruf einer einzelnen Frage innerhalb der Ergebnisansicht wird diese im Question Runner angezeigt. Die Darstellung der Frage und des Feedbacks entspricht dabei bewusst der des Übungsmodus (siehe Kapitel 3.2.3). Korrekte und falsche Antworten werden also entsprechend farblich hervorgehoben, die richtige Lösung sowie zugehörige Lösungswege werden angezeigt.

Durch die identische Darstellung des Feedbacks in Übungs- sowie Reviewmodus müssen sich Nutzer:innen nicht an eine neue Oberfläche gewöhnen. Dadurch wird die Orientierung erleichtert und die Prüfung besser als Lerneffekt fungieren, da Fehler direkt im bekannten Kontext nachvollzogen werden können.

Ist der/die Nutzer:in mit dem Analysieren der Fehler fertig, so kann der Reviewmodus jederzeit verlassen werden. Befindet sich der/die Nutzer:in auf der letzten Frage der Prüfung, so wird wie beim Prüfungsmodus der *"Weiter"*-Button durch einen *"Fertigstellen"*-Button ersetzt. Durch das Betätigen dieses Buttons wird der Reviewmodus beendet und der/die Nutzer:in zurück zur Prüfungskonfiguration geführt.

Alternativ kann diese Prüfung von hier aus auch neu gestartet werden. Dabei wird eine exakte Kopie der Prüfung mit den selben Fragen und dem selben Zeitlimit erstellt. Der/Die Nutzer:in kann somit die Prüfung beliebig oft neu starten, wenn das Verständnis für die eben gemachten Fehler direkt geprüft werden soll. Für diese Funktion ist direkt neben dem *"Weiter"*-Button ein Neustart-Symbol zu finden.



Abbildung 22: Neustart-Button im Reviewmodus

Damit ist der jeweilige Prüfungsdurchlauf vollständig abgeschlossen. Alle Ergebnisse bleiben gespeichert und können später erneut im Reviewmodus eingesehen werden,

ohne den Übungsmodus oder den Lernfortschritt zu beeinflussen. Wird eine Prüfung über die Neustart-Funktion erneut begonnen, so entsteht ein neuer, unabhängiger Prüfungsdurchlauf, während die Ergebnisse vorheriger Durchläufe weiterhin erhalten bleiben.

3.4 Use Case: Fortschritt einsehen & analysieren

Ein wichtiger Bestandteil unserer Lernplattform ist Fortschrittseinsicht, um den User die Möglichkeit zu bieten seinen aktuellen Wissensstand jeder Zeit einsehen zu können. Mit übersichtlichen Diagrammen und Statistiken hat der User immer einen klaren Überblick auf seinen Lernfortschritt und kann so effizient einschätzen wie viel Übungsbedarf besteht. Filteroptionen ermöglichen es auch nach verschiedenen Themenpools zu filtern damit Schüler gezielt erkennen in welchen Themenpools sie noch mehr Übung bräuchten.

Statistiken Ansicht

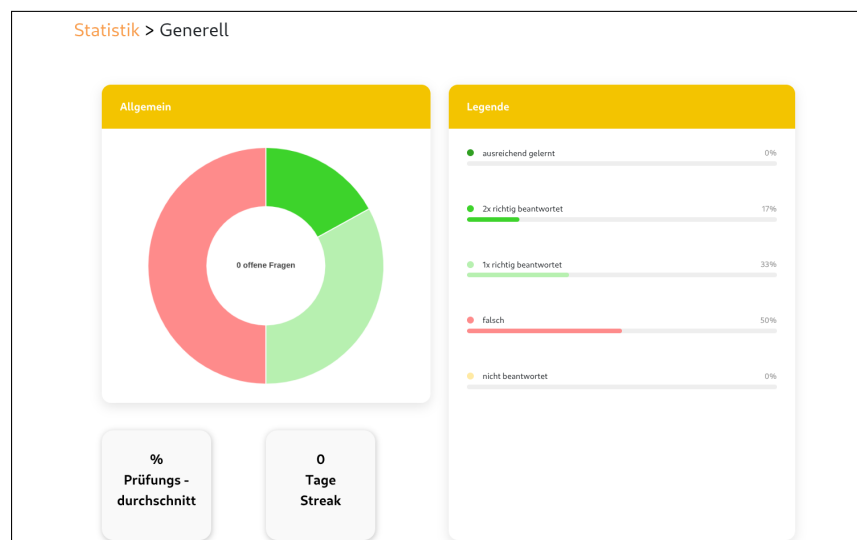


Abbildung 23: Allgemeine Statistikübersicht der Anwendung

Neben der Darstellung des Lernfortschritts mit Diagrammen sind Statistiken auch zur Motivation gut geeignet, ein sichtbarer Fortschritt motiviert für zukünftiges Lernen. Ein positives Lernerlebnis kann Nutzer auch für weitere Lernziele anspornen bzw. ihnen helfen regelmäßiger zu üben um ihre Leistung zu verbessern.

Aufgebaut sind die Statistiken und Diagramme auf den Ergebnissen und Daten aus dem Übungsmodus oder wenn der User eine Frage explizit übt. Die Auswahl der gestellten Fragen erfolgt über ein recherchiertes und ausgewähltes Lernsystem, um den Wissensstand langfristig im Kopf zu behalten.

3.4.1 Lernstrategien und Fehleranalyse

Identifikation von Wiederholungsbedarf und Schwachstellen.

Um den Lernerfolg langfristig zu verbessern, ist es notwendig, das Lernverhalten analysieren und Schwachstellen zu erkennen. Auf Grundlage der Fehleranalyse wird der Wiederholungsbedarf festgelegt. Da Nutzer:innen verstärkt die Fragen oder Themenpools üben die ihnen schwer fallen wird ihr Wissen und Sicherheit gestützt. Aufgaben und Fragen die sie bereits beherrschen und mehrfach korrekt beantwortet hatten, kommen hingegen in selteneren Abständen bzw. fallen zur gänze raus. Durch diese Anpassungen wird effizienter gelernt und somit auch mehr Lernerfolg erzielt.

Darüber hinaus unterstützt diese Funktion das selbstreflektierende Lernen. Die Nutzer:innen können nachvollziehen und selbst Muster erkennen, bei welchen Inhalten oder Fragen sie des öfteren mal Schwierigkeiten haben. Dadurch können die User selbst sehen wo sie Probleme haben und besser ihre Personalisierten Fragenpools auswählen, diese können sie anschließend üben.

Wie in Abbildung 23 gezeigt, gibt es unter der Hauptstatistik noch zwei Widgets. Die Streak-Anzeige und der Prüfungsdurchschnitt. Bei der Streak-Anzeige wird jedesmal wenn der User die Anwendung benutzt ein Tag hinzugefügt, wenn er die App an einem Tag nicht nutzt wird die Streak zurückgesetzt. Dafür wird das Datum des letzten Logins mit dem aktuellen Datum verglichen, wenn mehr als 24 Stunden seit dem letzten Login vergangen sind wird die Streak zurückgesetzt. So kann der/die Nutzer:in motiviert werden die App regelmäßig zu verwenden. Der Prüfungsdurchschnitt zeigt den Durchschnitt aller abgelegten Prüfungen an, dieser wird durch das Addieren aller erreichten Prozente geteilt durch die Anzahl der Prüfungen berechnet.

Prüfungsverlauf

Auf der Prüfungsverlaufansicht, wie in Abbildung 24 zu sehen ist, werden alle abgelegten Prüfungen des Nutzers angezeigt. Dafür werden alle Prüfungen aus dem Backend geholt und in einer Liste dargestellt. Jede Prüfung zeigt auf ersten Blick das Datum, die erreichten Prozente, das Fach und die Dauer an. Wenn man diese jedoch aufklappt kann der/die Nutzer:in jede Frage zur jeweiligen Prüfung sehen, dort werden ebenfalls weitere Details aufgelistet. Welche Antworten man zuvor gegeben hat und ob diese korrekt war, wie in Abbildung 25 gezeigt.

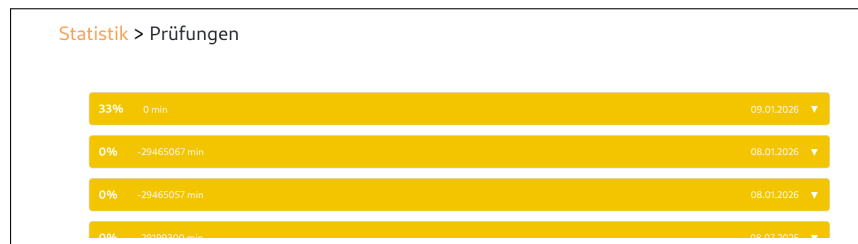


Abbildung 24: Prüfungsverlaufansicht

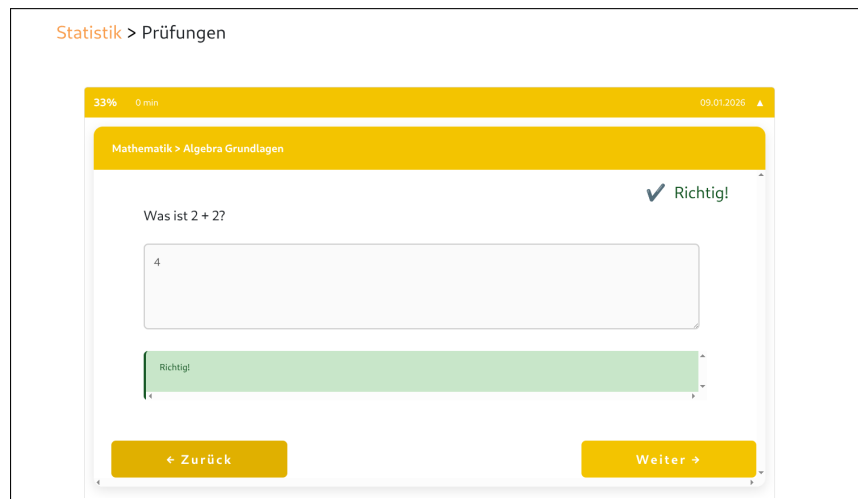


Abbildung 25: Prüfungsverlaufansicht Detail

3.4.2 Spaced Repetition nach Leitner

Technische Umsetzung der Lernstrategie, Speicherintervalle, Fortschrittslevel.

Die Grundlage für die Lernplattform bildet das Leitner-System in Kombination mit Spaced Repetition, die in Kapitel 1.5 beschrieben wurden. Die Lernmethoden bestehen aus 3 wichtigen Punkten

1. **Level:** Fragen werden in Level eingeteilt - je öfter richtig desto höher das Level
2. **Wiederholungsintervall:** Fragen werden gesperrt wenn man sie richtig hatte und erst nach einem gewissen Zeitraum entsperrt
3. **Anzeige:** Gesperrte und offene Fragen werden angezeigt Die Anzeige im Frontend dient dazu den

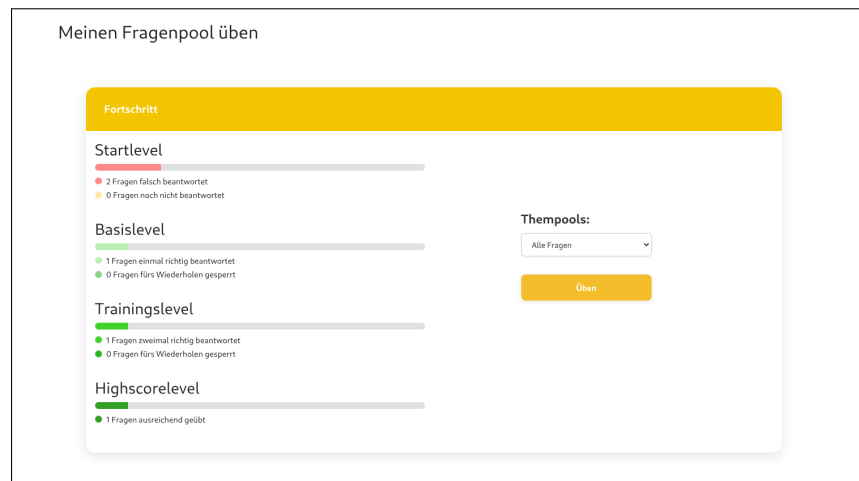


Abbildung 26: Anzeige der Startseite

In Abbildung 26, die die HomeComponentente zeigt bzw. die Umsetzung der Lernstrategien sieht man die verschiedenen "Level". Die Level sind nach dem Leitner-System aufgebaut, von denen es in diesem Fall vier gibt: Startlevel, Basislevel, Trainingslevel und Highscorelevel.

Jede Frage wird durch QuestionPoolEntry abgebildet, diese speichert alle relevante Daten für die Fehleranalyse oder das Wiederholen. Für das einteilen in die Level sind die Einträge lastAnsweredCorrectly und correctCount wichtig. LastAnsweredCorrectly merkt sich ob die Frage das letzte mal korrekt beantwortet wurden und correctCount wie oft du diese Frage bereits richtig hattest. Mithilfe einer Abfrage kann eingeordnet werden welche Frage in welches Level kommt. Es verläuft nach dem Schema

- **Startlevel:** Dort sind die falsch oder nicht beantworteten Fragen, diese sind immer zum üben offen.
- **Basislevel:** Hier befinden sich Fragen die einmalig richtig beantwortet waren, direkt danach sind sie für 1 Tag gesperrt.
- **Trainingslevel:** Fragen die der User 2x richtig hatte kommen ins Trainingslevel und sind nun 3 Tage gesperrt.
- **Highscorelevel:** Hier befinden sich Fragen die schon zur Genüge geübt wurden und somit kaum mehr geübt werden müssen.

Das Sperren erfolgt durch die Auswertung des Parameters answeredAt. So lässt sich nachvollziehen, wann der Nutzer die Frage zuletzt bearbeitet hat, ob die Antwort korrekt oder falsch war, in welchem Level sich die Frage zuvor befand bzw. wie hoch

der correctCount ist. Schlussendlich lässt sich dann sagen wie lange die Frist zum Wiederholen eingestellt wird.

Mit einen klick auf üben werden nicht nur irgendwelche Fragen ausgewählt, sondern genau die die der User am dringendsten benötigt. So zusagen die falsch oder nicht beantworteten Fragen zuerst, darauffolgend erst die mehrmals korrekt beantworteten Inhalte.

Die Anzeige im Frontend dient dazu den Nutzer:innen einen Überblick über ihren eigenen Lernfortschritt zu geben. Die einzelnen Level im Leitner-System werden als Progressbars visualisiert und mit Farben wird dem User sein Fortschritt symbolisiert.

3.4.3 Lernstatistiken und Visualisierung

Erhebung und grafische Darstellung der Daten zur Nutzerleistung. Bibliotheken und Darstellungskonzepte.

Visualisierung durch Chart.js

Die technische Umsetzung der Diagramme und Statistiken im Frontend erfolgt durch die Bibliothek Chart.js. Durch das Einbinden über die Bibliothek ng2-charts in Angular, können verschiedene Diagrammtypen einfach erstellt werden. Die meisten Diagramme in der Lernplattform sind Dounutdiagramme. In den Statistiken werden die Daten aus dem Backend abgefragt und in die Diagramme übergeben. Mittels correctCount werden die Fragen in verschiedene Kategorien eingeteilt, wie oft eine Frage richtig beantwortet wurde. Durch Dtos werden die Daten vom Backend richtig formatiert und im Frontend weiterverarbeitet. Die Unterteilung erfolgt in die Kategorien: nie beantwortet, einmal richtig, zweimal richtig und dreimal oder öfter richtig beantwortet. Die Daten werden dann dem Dounutdiagramme übergeben, die Prozente werden für die akurate Anzeige berechnet und mit Farben visualisiert.

Wie in Abbildung 23 zu sehen ist, befindet sich in der Mitte des Diagramms die Gesamtanzahl der Fragen, die der User bisher noch nicht beantwortet hat. Dies war jedoch keine Standardfunktion von Chart.js, sondern wurde individuell implementiert. Hierfür wurde ein Plugin erstellt, das einen Text im Zentrum des Diagramms rendert.

3.5 Implementierungsdetails

3.5.1 Datenmodell

Das Datenmodell bildet die grundlegende Struktur der gesamten Anwendung und wurde als gemeinsames Schema von beiden Entwicklungsteams entworfen. Es umfasst alle zentralen Entitäten zur Verwaltung von Nutzer:innen, Fragen, Themenbereichen, Übungsfortschritt und Prüfungssimulation.

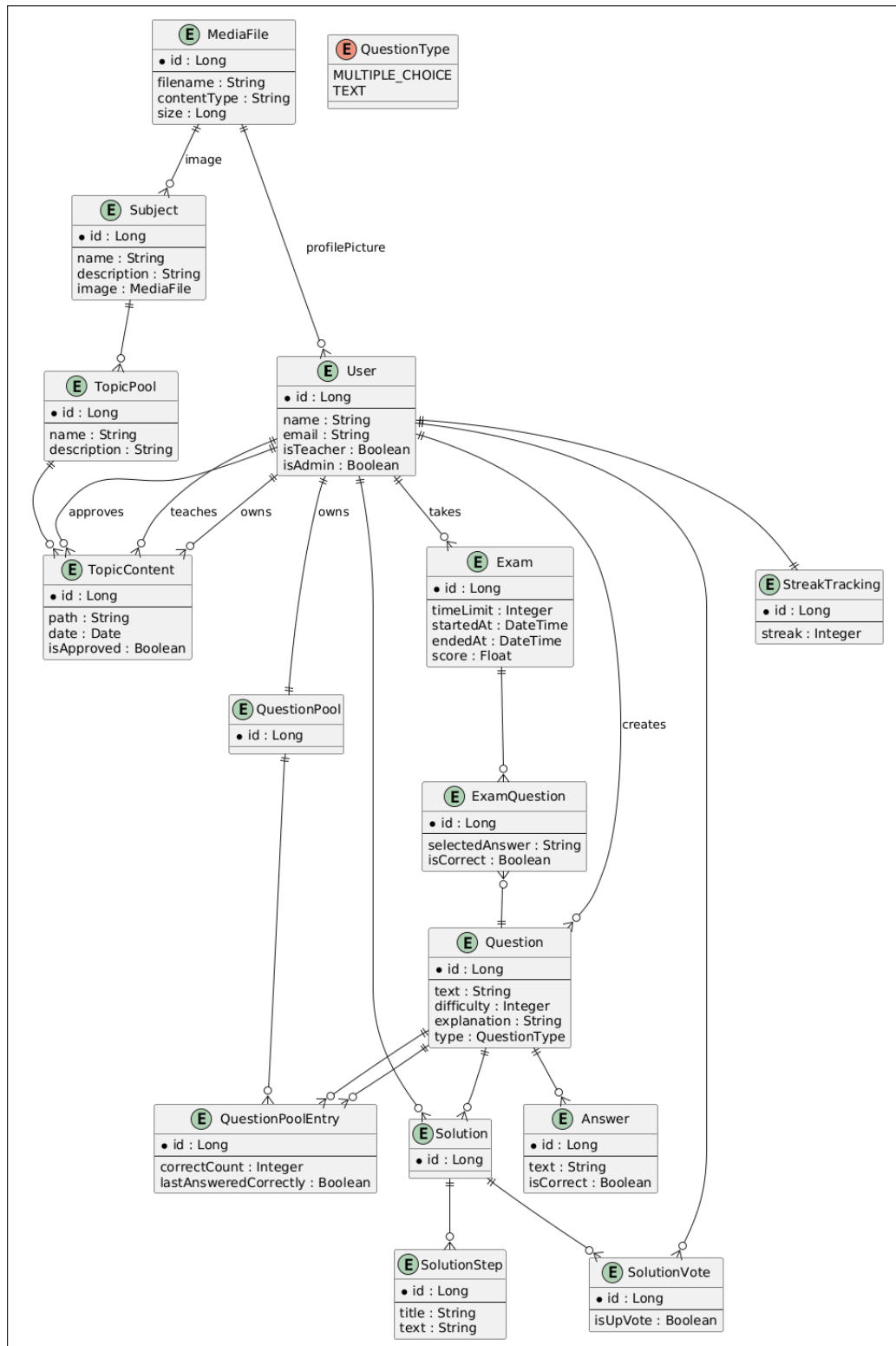


Abbildung 27: Datenbankmodell der Anwendung

Abbildung 27 zeigt das vollständige relationale Datenbankmodell der Anwendung. Nicht alle dargestellten Tabellen werden in jedem beschriebenen Use Case dieser Arbeit verwendet, da einzelne Komponenten von unterschiedlichen Teams umgesetzt wurden. Für die folgenden Abschnitte werden daher nur jene Entitäten näher betrachtet, die für die Umsetzung der Übungs- und Prüfungsfunktionen relevant sind.

Zentrale Entitäten für den Übungsmodus

Für den Übungsmodus sind insbesondere die Entitäten *Question*, *Answer*, *QuestionPool*, *QuestionPoolEntry* und *User* relevant. Sie bilden die Grundlage, um Fragen darzustellen, Antworten zu erfassen und den Lernfortschritt der Nutzer:innen nachzuvollziehen.

Question

Die Entität *Question* repräsentiert eine einzelne Übungsfrage. Sie enthält den Fragetext und den Fragetyp (*MULTIPLE CHOICE*, *TEXT*). Jede Frage kann mehrere Antworten besitzen, die in der Entität *Answer* hinterlegt sind.

Answer

Answer-Objekte beinhalten den Antworttext und einen Eintrag, ob die Antwort korrekt ist (*isCorrect*). Unabhängig vom Fragetypen werden alle richtigen und falschen Antworten auf eine Frage gespeichert. Bei Multiple-Choice-Fragen müssen alle richtigen und keine falschen Antworten angegeben werden, um als korrekt gewertet zu werden. Für Freitext-Fragen werden nur richtige Antworten angelegt. Die eingegebene Antwort muss hierbei einer der Einträge gleichen, um korrekt gewertet zu werden.

QuestionPool und QuestionPoolEntry

Der individuelle Fragenpool der Nutzer:innen wird von einem *Questionpool*-Objekt repräsentiert. Die Entität *QuestionPoolEntry* verbindet eine Frage mit einem Pool und speichert den individuellen Fortschritt des Nutzers/der Nutzerin. Dazu zählen unter anderem:

- *correctCount*: Anzahl der aufeinanderfolgend korrekt geantworten Male für diese Frage
- *lastAnsweredCorrectly*: ob die Frage zuletzt richtig beantwortet wurde
- *answeredAt*: Zeitpunkt, zu dem die Frage zuletzt beantwortet wurde

Diese Tabelle ermöglicht die Umsetzung der Wiederholungslogik im Übungsmodus, indem Fragen je nach Bearbeitungsstatus und Lernerfolg wiederholt oder vorübergehend gesperrt werden.

User

Die Entität *User* enthält die grundlegenden Informationen über die Lernenden. Sie ist mit den *QuestionPool*-Objekten verbunden, um den individuellen Lernfortschritt zu

speichern, sowie mit *Question* und *Solution*, um erstellte Fragen und Lösungswege zu verwalten.

Funktionale Zusammenhänge

In Abbildung 27 sind die Beziehungen zwischen diesen Entitäten dargestellt. Zusammen ermöglichen sie:

- Die flexible Auswahl von Fragen aus einem Pool für eine Übungssitzung
- Das Tracking des individuellen Lernfortschritts pro Frage
- Die Umsetzung von Wiederholungs- und Sperrlogiken im Übungsmodus

Diese Struktur ermöglicht, dass im Übungsmodus flexibel auf die Bedürfnisse der Nutzer:innen eingegangen werden kann, während gleichzeitig eine saubere Trennung zwischen Fragen, Fragenpools der Nutzer:innen und individuellen Fortschritten besteht.

Zentrale Entitäten für Lösungswege

Für die Darstellung von Lösungswegen im Übungsmodus sind vor allem die Entitäten *Solution*, *SolutionStep* und *SolutionVote* relevant. Durch diese können Nutzer:innen nicht nur die richtige Lösung sehen, sondern auch die einzelnen Schritte nachvollziehen und Feedback von anderen Nutzer:innen erhalten. Zudem können individuelle Lösungswege hochgeladen werden, um sich und anderen Nutzer:innen in der Zukunft zu unterstützen.

Solution

Die Entität *Solution* repräsentiert einen Lösungsweg zu einer Frage. Jede Lösung gehört zu genau einer Frage (*Question*) und einem erstellenden Nutzer (*User*) und stellt sich aus mehreren *SolutionStep*-Objekten zusammen

SolutionStep

SolutionStep-Objekte enthalten die einzelnen Schritte einer Lösung. Jeder Schritt besitzt einen Titel zur Zusammenfassung und einen Text zur Erklärung und Veranschaulichung des nächsten Schritts. Durch die Schritt-für-Schritt-Darstellung kann der/die Nutzer:in den Lösungsweg nachvollziehen und gezielt lernen.

SolutionVote

Diese Entität ermöglicht es anderen Nutzer:innen, eine Lösung positiv oder negativ zu bewerten (*isUpVote*). So werden Lösungswege hochwertige Lösungswege mit mehr

positiven Bewertungen hervorgehoben und die Community kann gegenseitig Feedback geben.

Funktionale Zusammenhänge

- Jede Frage kann mehrere von Nutzer:innen erstellte Lösungen besitzen.
- Jede Lösung besteht aus mehreren Schritten.
- Nutzer:innen können Lösungswege bewerten, wodurch die Qualität sichtbar wird.
- Die Anzeige der Lösungen erfolgt im Übungsmodus direkt nach der Beantwortung, wodurch der Lernprozess unterstützt wird.

Datenmodell für Prüfungssimulationen

Für die Umsetzung des Prüfungsmodus sind relevante Entitäten vor allem *Exam* und *ExamQuestion*. Durch diese sind die Verwaltung von Prüfungen, die Speicherung der Antworten und die nachträgliche Auswertung möglich.

Exam

Die Entität *Exam* stellt eine einzelne Prüfung dar. Sie enthält Informationen über das Zeitlimit, den Beginnzeitpunkt, den Endzeitpunkt sowie dem Gesamtergebnis. Über die Beziehung zu *User* ist klar, welcher Nutzer/welche Nutzerin diese Prüfung abgelegt hat. Jede Prüfung kann mehrere Fragen enthalten, die über die Entität *ExamQuestion* abgebildet werden.

ExamQuestion

ExamQuestion speichert die gegebene Antworten auf die einzelne Frage innerhalb einer Prüfung und ist mit dem dementsprechendem Fragenobjekt verknüpft. Sie speichert nicht nur die ausgewählte Antwort, sondern auch ob diese Frage korrekt beantwortet wurde. So können alle Fragen einer Prüfung individuell ausgewertet werden und im Reviewmodus angezeigt werden.

Funktionale Zusammenhänge

Durch die Beziehung zwischen *Exam* und *ExamQuestion* wird folgendes ermöglicht:

- Speicherung von Prüfungen für einzelne Nutzer:innen
- Zuordnung der ausgewählten Antworten zu den jeweiligen Fragen

- Berechnung des Gesamtergebnisses und der prozentual korrekt beantworteten Fragen
- Nachträgliche Ansicht der Prüfung im Reviewmodus, inklusive der Anzeige von korrekt und falsch beantworteten Fragen

Diese Struktur macht eine realistische Simulation einer Prüfungssituation möglich. Die Nutzer:innen bearbeiten Fragen unter Zeitdruck, ihre Antworten werden erfasst und persisitiert und eine nachträgliche Auswertung ist jederzeit möglich.

3.5.2 Überblick über Komponenten

Backend / API

Das auf Quarkus basierende Backend stellt über REST-Endpunkte die zentrale Schnittstelle für das Frontend bereit. Dabei werden alle Daten über JSON-Format übertragen, wobei klassische HTTP-Methoden wie *GET* und *POST* verwendet werden.

Die Kernaufgaben des Backends umfassen:

- **Fragenmanagement:**
Laden von Fragen, Abrufen von Antwortoptionen und Speichern der Nutzerantworten. Die Logik zur Bewertung der Antworten wird durch Services wie *AnswerService* umgesetzt.
- **Prüfungslogik:**
Erstellung und Verwaltung von Prüfungen über den *ExamService*. Dieser Service wählt Fragen aus den individuellen Fragenpools aus, setzt Zeitlimits und berechnet den Endpunktestand. Auch Funktionen wie das Kopieren bereits abgeschlossener Prüfungen werden hier ermöglicht.
- **Fortschritts- und Statistikfunktionen:**
Der *StatsService* liefert alle Daten für die Fortschritts- und Statistikübersichten im Frontend. Dazu gehören globale Statistiken, Fortschritte in einzelnen Themenbereichen sowie die Umsetzung der Sperrlogik für Übungsfragen, um Wiederholungen strukturiert zu steuern.
- **Kommunikation mit dem Frontend:**
Das Backend stellt alle relevanten Daten für den *Question Runner*, die Fortschrittsübersicht und die Ergebnisanzeige der Prüfungen bereit. Eingehende Antworten werden validiert, ausgewertet und in der Datenbank persistiert.

- **Datenzugriff:**

Services greifen auf die Daten über dedizierte Repository-Klassen (wie zum Beispiel *QuestionRepository* oder *ExamRepository*) zu. Das führt zu einer klaren Trennung zwischen Geschäftslogik und Datenzugriff.

Das Backend ist unter anderem für besondere Funktionen, wie die Wiederholungslogik für Übungsfragen, die Sicherstellung der Prüfungskonsistenz sowie die Kopie von abgeschlossenen Prüfungen, verantwortlich. Dadurch bleibt die Trennung zwischen Übungs- und Prüfungsmodus sauber erhalten und im Frontend kann die Darstellung vollständig im Fokus stehen.

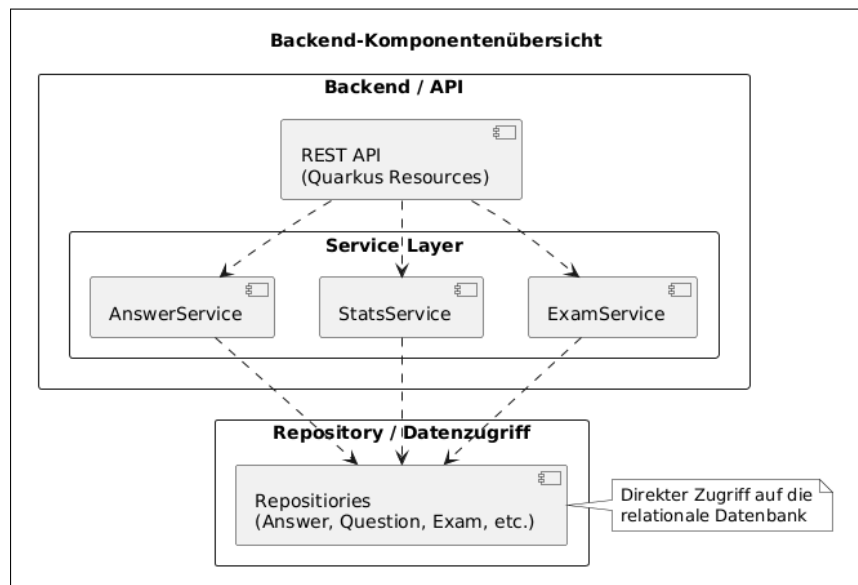


Abbildung 28: Komponentendiagramm des Backends

Frontend-Komponenten

Das Frontend der Anwendung ist sowohl für die Darstellung der Inhalte als auch für die Interaktion der Nutzer:innen mit dem Übungs- und Prüfungsmodus verantwortlich. Dabei ist eine klare Struktur und Wiederverwendung zentraler Komponenten Grundlage für die Gewährleistung von konsistenten Nutzererlebnissen über alle Modi hinweg.

Eine zentrale Rolle nimmt dafür der **Question Runner** ein. Diese Komponente wird sowohl im Übungs-, Prüfungs- als auch im Reviewmodus verwendet und ist für die Darstellung einzelner Fragen, der zugehörigen Antwortmöglichkeiten sowie der Navigation zwischen Fragen verantwortlich. Je nachdem, welcher Modus aktiv ist, verändert sich dabei das Verhalten der Komponente, in Bezug auf Feedback, Darstellung des Zeitlimits oder Bearbeitbarkeit von Antworten, während die grundlegende Oberfläche unverändert bleibt.

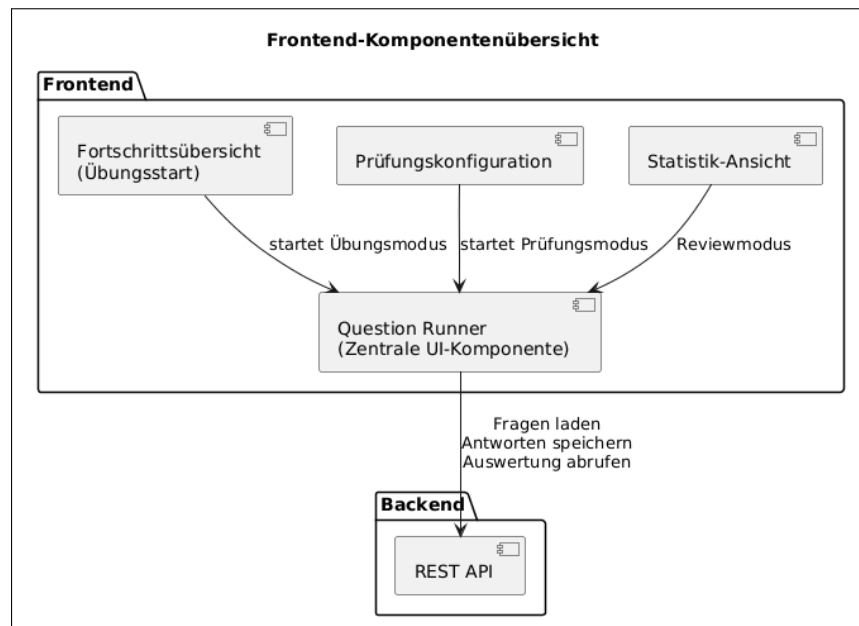


Abbildung 29: Komponentendiagramm des Frontends

In die jeweiligen Modi kann über eigene Views eingestiegen werden, wie etwa die Fortschrittsübersicht im Übungsmodus oder die Prüfungsconfiguration im Prüfungsmodus. Diese Komponenten sind vor allem für die Auswahl von Themenbereichen und den Start eines Durchlaufs zuständig und übergeben lediglich die notwendigen Parameter an den Question Runner.

Diese Aufteilung gewährleistet eine klare Trennung zwischen Darstellungslogik und fachlicher Logik. Zur gleichen Zeit ermöglicht dieser modulare Aufbau eine einfache Erweiterbarkeit für weitere Modi, ohne bestehende Komponenten grundlegend anpassen zu müssen.

3.5.3 Custom CSS-Klassen und eingebundene Bibliotheken

Die Implementierung der Lernplattform erfordert sowohl die Integration externer Bibliotheken zur Datenvisualisierung als auch die Entwicklung wiederverwendbarer CSS-Komponenten. Diese Kombination gewährleistet eine effiziente Entwicklung bei gleichzeitig konsistentem Erscheinungsbild der Anwendung.

Visualisierung von Statistiken mit Chart.js

Um den Lernfortschritt der Nutzer:innen übersichtlich und ansprechend darzustellen, wird im Frontend die JavaScript-Bibliothek Chart.js verwendet. Diese Open-Source-Lösung hat sich als Industriestandard für webbasierte Datenvisualisierungen etabliert

und bietet eine Vielzahl an Diagrammtypen, die individuell angepasst und mit dynamischen Daten befüllt werden können.

Die Wahl von Chart.js begründet sich durch mehrere technische und praktische Vorteile: Die Bibliothek ist weit verbreitet, gut dokumentiert und lässt sich nahtlos in das Angular-Framework integrieren. Zudem bietet sie eine reaktive API, die eine einfache Aktualisierung von Diagrammen bei Datenänderungen ermöglicht. In der entwickelten Lernplattform werden hauptsächlich Donut- und Balkendiagramme verwendet, um verschiedene Aspekte der Lernkurve aufzuzeigen (siehe Abbildung 23).

Implementierung von Donutdiagrammen

Donutdiagramme eignen sich besonders zur Darstellung prozentualer Verteilungen, beispielsweise des Bearbeitungsstatus von Lernkarten oder der Verteilung von Fachgebieten im Lernfortschritt. Die ringförmige Darstellung mit zentralem Freiraum ermöglicht zusätzlich die Platzierung von Gesamtstatistiken oder Schlüsselkennzahlen im Zentrum des Diagramms.

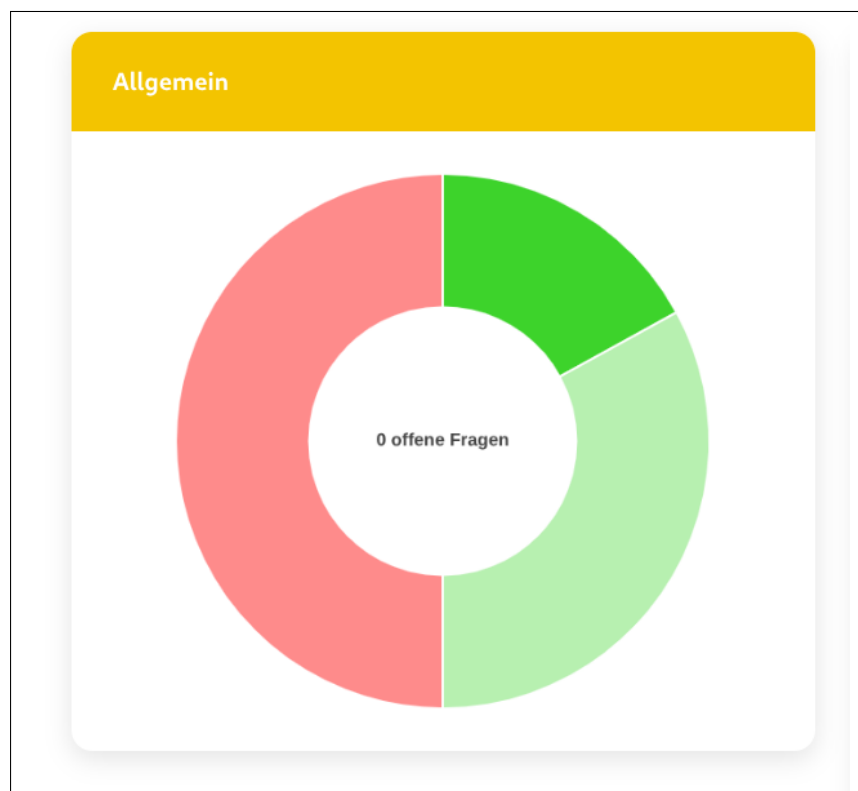


Abbildung 30: Beispielhafte Implementierung von Chart.js zur Visualisierung des Lernfortschritts

Technische Aspekte der Chart.js-Integration

- **Datenaktualisierung:** Die dynamische Natur der Lernplattform erfordert häufige Aktualisierungen der Diagramme, wenn neue Lerndaten generiert werden. Chart.js stellt hierfür die Methode `update()` bereit, welche eine effiziente Neuberechnung und Neudarstellung des Diagramms ermöglicht, ohne dass die gesamte Seite neu geladen werden muss. Dies verbessert die User Experience erheblich, da Übergänge flüssig animiert werden und der Anwendungszustand erhalten bleibt.

Bei der Implementierung wird darauf geachtet, dass nur die tatsächlich geänderten Datenpunkte aktualisiert werden, was die Performance bei größeren Datensätzen optimiert. Die Bibliothek übernimmt dabei automatisch die Interpolation zwischen alten und neuen Werten und erzeugt so eine visuell ansprechende Übergangsanimation.

- **Anpassung des Designs:** Chart.js bietet umfangreiche Möglichkeiten zur Anpassung des visuellen Designs der Diagramme. Farben, Schriftarten, Abstände und andere Stilelemente können so konfiguriert werden, dass sie zum Gesamtdesign der Anwendung passen und ein kohärentes Erscheinungsbild gewährleisten.

In der Lernplattform werden die Farben der Diagramme an das definierte Farbschema der Anwendung angepasst, wobei unterschiedliche Lernstatus durch verschiedene Farbtöne repräsentiert werden. Beispielsweise werden erfolgreich absolvierte Lerneinheiten in Grüntönen, noch zu bearbeitende Inhalte in neutralen Grautönen und fehlerhafte Versuche in Rottönen dargestellt. Diese konsistente Farbkodierung unterstützt die intuitive Erfassung des Lernstatus.

- **Benutzerdefinierte Plugins:** Für spezielle Anforderungen, die über die Standardfunktionalität von Chart.js hinausgehen, können benutzerdefinierte Plugins entwickelt werden. Diese ermöglichen die Implementierung zusätzlicher Funktionalitäten, wie beispielsweise:
 - Anzeige von benutzerdefinierten Tooltips mit erweiterten Informationen beim Hovern über Datenpunkte
 - Integration von Schwellwertlinien zur Markierung von Lernzielen oder Mindestanforderungen
 - Implementierung von Exportfunktionen zur Speicherung von Diagrammen als Bilddateien
 - Hinzufügen von interaktiven Legenden mit erweiterten Filterfunktionen

Die Plugin-Architektur von Chart.js ist so konzipiert, dass eigene Erweiterungen nahtlos in den Rendering-Prozess integriert werden können, ohne die Kernfunktionalität der Bibliothek zu beeinträchtigen.

Wiederverwendbare CSS-Klassen

Die Anwendung nutzt ein System wiederverwendbarer CSS-Klassen, um ein konsistentes Design und Layout über alle Komponenten hinweg zu gewährleisten. Diese Klassen sind in einer zentralen, geteilten CSS-Datei definiert und können komponentenübergreifend verwendet werden. Dieser Ansatz folgt dem DRY-Prinzip (Don't Repeat Yourself) und bringt mehrere Vorteile mit sich:

- **Konsistenz:** Ein einheitliches Erscheinungsbild der Anwendung wird durch die Verwendung derselben Stilklassen in verschiedenen Komponenten sichergestellt.
- **Wartbarkeit:** Änderungen am Design müssen nur an einer zentralen Stelle vorgenommen werden und wirken sich automatisch auf alle verwendenden Komponenten aus.
- **Teamarbeit:** Die gemeinsame CSS-Datei vereinfacht die Zusammenarbeit im Team, da alle Entwickler:innen auf denselben Stil-Definitionen aufbauen.
- **Reduzierung von Redundanz:** Durch die Wiederverwendung werden doppelte Code-Definitionen vermieden, was die Dateigröße reduziert und die Ladezeit der Anwendung optimiert.

Beispiele häufig verwendeter CSS-Klassen

Die folgenden Klassen bilden das Grundgerüst des visuellen Designs der Lernplattform:

- **.widget-wrapper:** Diese Klasse definiert den Grundcontainer für die Widget-Darstellung der Anwendung. Widgets sind modulare UI-Komponenten, die Informationen kompakt und übersichtlich präsentieren, wie beispielsweise Statistikkarten, Fortschrittsanzeigen oder Lernziel-Übersichten.

Die `.widget-wrapper`-Klasse sorgt für ein einheitliches Layout aller Widgets durch standardisierte Abstände, Padding-Werte, Border-Radius und Schatten-Effekte. Dies gewährleistet, dass verschiedene Widget-Typen visuell harmonisieren und als zusammengehörige Elemente wahrgenommen werden. Zusätzlich implementiert die

Klasse responsive Breakpoints, sodass Widgets auf verschiedenen Bildschirmgrößen optimal dargestellt werden.

- **.header:** Die Header-Klasse fügt, wo benötigt, einen visuell hervorgehobenen Kopfbereich hinzu, der mit der primären Farbe des Anwendungsdesigns gestaltet ist. Diese Klasse wird typischerweise für Überschriften von Widgets, Sektionen oder modalen Dialogen verwendet.

Die primäre Designfarbe schafft dabei eine klare visuelle Hierarchie, sodass Nutzer:innen wichtige Bereiche der Anwendung schnell erkennen können. Neben der Hintergrundfarbe legt die Klasse auch Textfarbe, Schriftgröße, Schriftgewicht und vertikale Abstände fest, damit alle Header-Elemente einheitlich dargestellt werden.

- **button.submit-btn:** Diese Klasse stellt einen standardisierten Button-Stil für Interaktionen bereit und kommt vor allem bei Übungsaufgaben, Formular-Submissions und anderen Bestätigungsaktionen zum Einsatz.

Der `.submit-btn` zeigt visuelles Feedback für unterschiedliche Zustände: normale Darstellung, Hover-Effekt, aktiver Zustand beim Klicken und deaktivierter Zustand. Nutzer:innen können dadurch schnell erkennen, welche Schaltflächen primäre Aktionen auslösen. Übergänge und Animationen sorgen dafür, dass Interaktionen flüssiger wirken.

Die Button-Klasse berücksichtigt auch Accessibility-Aspekte wie ausreichende Größe für Touch-Bedienung, gute Farbkontraste und sichtbare Fokus-Indikatoren bei Tastatur-Navigation.

CSS-Architektur und Best Practices

Die CSS-Klassen sind modular strukturiert und unterscheiden zwischen globalen Basis-klassen, komponentenspezifischen Stilen und Utility-Klassen. So lassen sich einzelne Komponenten flexibel anpassen, ohne dass globale Stile beeinflusst werden. Außerdem treten dadurch weniger Spezifität-Konflikte auf, was die Wartbarkeit des Codes verbessert.

4 Zusammenfassung

Literaturverzeichnis

- [1] Ajay Kumar S, „Evolution of Spring Boot and Microservices,” letzter Zugriff am 01.03.2026. Online verfügbar: <https://medium.com/javarevisited/evolution-of-spring-boot-and-microservices-4d1109b5a4d3>
- [2] Michael Vitz, „Java and its annotations,” letzter Zugriff am 02.03.2026. Online verfügbar: <https://www.innoq.com/en/articles/2024/09/java-annotations/>
- [3] —, „Was ist die Magie von Spring Boot?” letzter Zugriff am 01.03.2026. Online verfügbar: <https://www.innoq.com/de/articles/2020/02/spring-boot-magie/>
- [4] Viprav Fauzdar, „Quarkus vs Spring Boot — The Cloud-Native Java Showdown for 2025,” letzter Zugriff am 02.03.2026. Online verfügbar: <https://medium.com/@viprav.fauzdar/quarkus-vs-spring-boot-the-cloud-native-java-showdown-for-2025-deb8bd990894>
- [5] Baeldung, „Spring Boot with Multiple SQL Import Files,” letzter Zugriff am 02.03.2026. Online verfügbar: <https://www.baeldung.com/spring-boot-sql-import-files>
- [6] Ian Holton, „Node.js - Die fünf wichtigsten Fakten,” letzter Zugriff am 02.03.2026. Online verfügbar: https://www.esono.de/blog/softwareentwicklung/node-js-die-fuenf-wichtigsten-fakten_aid_1127.html
- [7] OpenJS Foundation, „The Node.js Event Loop,” letzter Zugriff am 09.03.2026. Online verfügbar: <https://nodejs.org/en/learn/asynchronous-work/event-loop-timers-and-nexttick>
- [8] Muhammad Sufiyan, „Improving Performance in Node.js: Blocking the Event Loop,” letzter Zugriff am 09.03.2026. Online verfügbar: <https://innosufiyan.hashnode.dev/improving-performance-in-nodejs-blocking-the-event-loop>
- [9] ElAmit Mansour, „The Node.js “Single-Threaded” Myth: The 5-Thread Truth Hiding in Plain Sight,” letzter Zugriff am 09.03.2026. Online verfügbar: <https://elamir.medium.com/the-node-js-single-threaded-myth-the-5-thread-truth-hiding-in-plain-sight-db4b9be3c099>
- [10] OpenJS Foundation, „Don’t Block the Event Loop (or the Worker Pool,” letzter Zugriff am 09.03.2026. Online verfügbar: <https://nodejs.org/en/learn/asynchronous-work/dont-block-the-event-loop>
- [11] —, „About Node.js,” letzter Zugriff am 09.03.2026. Online verfügbar: <https://nodejs.org/en/about>
- [12] Prisma Data Inc., „What is Prisma ORM?” letzter Zugriff am 09.03.2026. Online verfügbar: <https://www.prisma.io/docs/orm>
- [13] TypeORM, „TypeORM - Code with Confidence. Query with Power.” letzter Zugriff am 09.03.2026. Online verfügbar: <https://typeorm.io/>

- [14] Ashley Davis, „A complete guide to full-stack live reload,” letzter Zugriff am 09.03.2026. Online verfügbar: <https://blog.logrocket.com/complete-guide-full-stack-live-reload/>
- [15] Yatin Batra, „How to Create Database Seed Scripts in Node.js,” letzter Zugriff am 09.03.2026. Online verfügbar: <https://www.javacodegeeks.com/how-to-create-database-seed-scripts-in-node-js.html>
- [16] ClearPeeks, „Configuration Management in Node.js,” letzter Zugriff am 09.03.2026. Online verfügbar: <https://www.clearpeaks.com/configuration-management-in-node-js/>
- [17] IBM, „The JIT compiler,” letzter Zugriff am 14.03.2026. Online verfügbar: <https://www.ibm.com/docs/en/sdk-java-technology/8?topic=reference-jit-compiler>
- [18] Red Hat, „Quarkus — Supersonic Subatomic Java,” letzter Zugriff am 03.01.2026. Online verfügbar: <https://quarkus.io>
- [19] ERP Solutions oodles, „The Pros and Cons of Quarkus vs Spring Boot,” letzter Zugriff am 14.03.2026. Online verfügbar: <https://erpsolutionsoodles.medium.com/the-pros-and-cons-of-quarkus-vs-spring-boot-a38e8b8df479>
- [20] Red Hat, „Quarkus Guide – Re-augment a Quarkus Application,” letzter Zugriff am 03.01.2026. Online verfügbar: <https://quarkus.io/guides/reaugmentation>
- [21] —, „Quarkus - Extensions,” letzter Zugriff am 03.01.2026. Online verfügbar: <https://quarkus.io/extensions/>
- [22] —, „Dev Services Overview,” letzter Zugriff am 14.03.2026. Online verfügbar: <https://quarkus.io/guides/dev-services>
- [23] Daniel Oh, „Simplify Java persistence using Quarkus and Hibernate Reactive,” letzter Zugriff am 14.03.2026. Online verfügbar: <https://developers.redhat.com/articles/2022/01/06/simplify-java-persistence-using-quarkus-and-hibernate-reactive>
- [24] David, „Speed up development and save time with Live Reload and Testing in Quarkus 2.0,” letzter Zugriff am 14.03.2026. Online verfügbar: <https://medium.com/geekculture/speed-up-development-and-save-time-with-live-reload-and-testing-in-quarkus-2-0-53af6ff65edb>
- [25] Red Hat, „How dev mode differs from a production application,” letzter Zugriff am 14.03.2026. Online verfügbar: <https://quarkus.io/guides/dev-mode-differences>
- [26] Arnaud Roques, „PlantUML,” letzter Zugriff am 05.01.2026. Online verfügbar: <https://www.plantuml.com/plantuml/uml/>

Abbildungsverzeichnis

1	Logo von Spring Boot	10
2	Logo von Node.js	14
3	Logo von Quarkus	19
4	Logo von Angular	27
5	Logo von Angular Material	28
6	Logo von Flutter	30
7	Ablaufdiagramm eines Users	34
8	Ansicht der Verwaltung des Fragenpools	36
9	Beispielhafte Ansicht einer Frage im Question Runner	40
10	Question-Runner: Beispiele für verschiedene Fragetypen	42
11	Fortschrittsübersicht des Übungsmodus	43
12	Auswahl des Themenbereichs für den nächsten Übungsdurchlauf	44
13	Question-Runner: Richtig und Falsch beantwortete Frage	45
14	"PrüfenButton wurde zum AbschließenButton	46
15	Seitennavigationsleiste bei ausgewähltem Prüfungsmodus	47
16	Beispielhafte Ansicht der Prüfungs-Konfiguration	48
17	Start der Prüfung nach Konfiguration	49
18	Prüfungsmodus: Navigationsarten	50
19	Prüfungsmodus: Letzte Frage - Fertigstellen-Button	50
20	Ergebnisübersicht nach Abschluss der Prüfung	52
21	Navigationsleiste nach Auswertung der Prüfung	53
22	Neustart-Button im Reviewmodus	53
23	Allgemeine Statistikübersicht der Anwendung	54
24	Prüfungsverlaufansicht	56
25	Prüfungsverlaufansicht Detail	56
26	Anzeige der Startseite	57
27	Datenbankmodell der Anwendung	60
28	Komponentendiagramm des Backends	65
29	Komponentendiagramm des Frontends	66
30	Beispielhafte Implementierung von Chart.js zur Visualisierung des Lernfortschritts	67

Tabellenverzeichnis

1	Vergleichende Bewertung der untersuchten Lernstrategien	6
2	Gegenüberstellung der Vor- und Nachteile von Angular	29
3	Gegenüberstellung der Vor- und Nachteile von Flutter	31
4	Direkter Vergleich der Kriterienbewertung zwischen Angular und Flutter	33